

Session 3.1

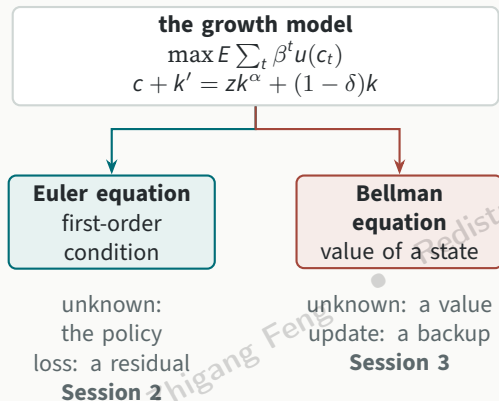
The Growth Model, With One Thing Hidden

Dynamic programming works, and says exactly what it needs; this deck is what happens when it cannot have it



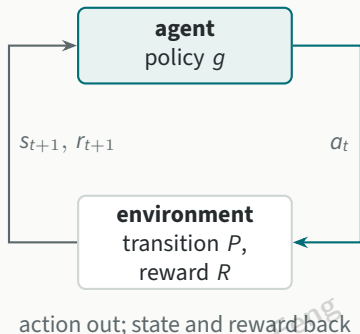
Zhigang Feng

Session 2 Took the Euler Road. The Bellman Equation Is the Other One



- ▶ Session 2 trained a policy network on Euler residuals. It never approximated a value.
- ▶ The Bellman equation describes the same optimum through the *value* of each state.
- ▶ It needs no derivatives: discrete choices and binding constraints are fine.
- ▶ **This session takes that road, and asks what survives when the model cannot be written down.**

Learning by Acting: the Third Way to Learn



- ▶ **Reinforcement learning:** act, observe a reward and a new state, adjust the rule.
- ▶ Session 2 had a label at every input. Here there is only a *reward*: one number, often late.
- ▶ **Credit assignment:** which of many past choices earned it?
- ▶ **Every algorithm in this session answers that question.**

Source: R. Sutton, A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., 2018, ch. 1 and 3.

Session 3.1

The Benchmark, and What It Requires

Dynamic programming works, and says exactly what it needs



The Growth Model: a Sequence Problem, and Its Recursive Form

sequence problem

$$\max E_0 \sum_t \beta^t \ln c_t$$

$$c_t + k_{t+1} = y(k_t, z_t)$$

one function, not a path

Bellman equation

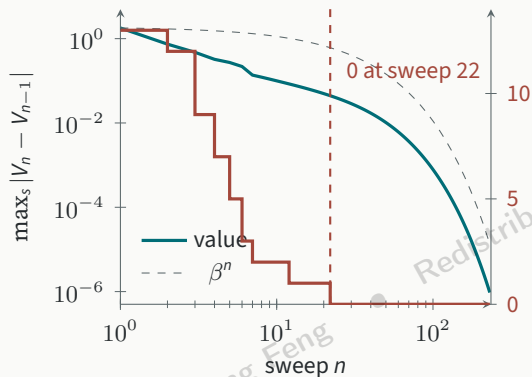
$$V = \max_{k'} \{ \ln c + \beta E[V' \mid z] \}$$

$$y(k, z) = zk^\alpha + (1 - \delta)k: \text{output plus capital left}$$

- ▶ k is a *real number*. So is the choice: next capital k' , anywhere in $(0, y)$.
- ▶ Equivalently a **saving share** $a \in (0, 1)$, with $k' = ay$ and $c = (1 - a)y$ — positive consumption at every a , which is Session 2's device.
- ▶ Two shocks only, $z \in \{0.9, 1.1\}$, each staying with probability 0.8.
- ▶ **Nothing here mentions a grid. The grid arrives with the algorithm, in two frames' time.**

Source: Miao & Feng, *AI for Economists*, ch. 11, §11.1. Calibration throughout Session 3 is the book's — $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2 — not Sessions 1–2's $\alpha = 0.33$, $\beta = 0.96$.

The Benchmark: Dynamic Programming, as Session 1 Left It



- ▶ Session 1.4: T is a contraction of modulus β , so **value function iteration** finds its one fixed point.
- ▶ Red, right axis: states where the greedy rule $g_n = \arg \max_{k'} W(k')$ is still wrong — 13, 12, 9, 7, 5, 3, then two for five sweeps and *one* for ten more, zero at 22. The value needs 231 sweeps.
- ▶ **The residual is not the policy. Deck 3.6 shows the converse, and it is worse.**

Source: Session 3 lab, block A; 7×2 grid, $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2.

Value Function Iteration, Line by Line

Value function iteration on a grid $\mathcal{K} \times \mathcal{Z}$

```
1 initialise  $V_0(k, z) \leftarrow 0$ 
2 repeat
3   for every grid point  $(k, z)$ :
4     for every  $k' \in \mathcal{K}$  with  $c = zk^\alpha + (1 - \delta)k - k' > 0$ :
5       score it:  $W(k') \leftarrow u(c)$ 
6       add tomorrow:  $W(k') \leftarrow W(k') + \beta \sum_{z'} P_z(z' | z) V_n(k', z')$ 
7      $V_{n+1}(k, z) \leftarrow \max_{k'} W(k')$ ,  $g_{n+1}(k, z) \leftarrow \arg \max_{k'} W(k')$ 
8 until  $\max_{(k, z)} |V_{n+1} - V_n| < \text{tol}$ 
```

- ▶ **Line 4** decides which actions exist (f, δ); **line 5** scores one (u); **line 6** averages over tomorrow (P_z). Three reads, three jobs — and **sampling removes only line 6's sum**.
- ▶ **Line 3 is the grid: DP discretises the state because it must sweep it.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “Value function iteration: how DP computes”.

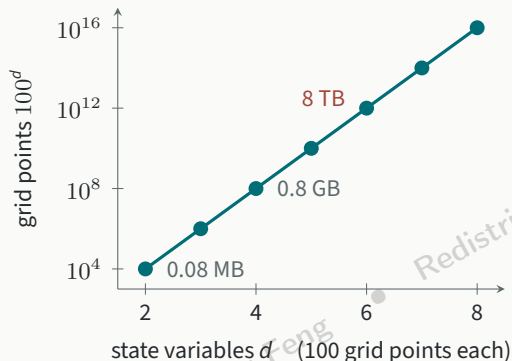
What Dynamic Programming Quietly Requires

Requirement	In the algorithm	Where it fails
to enumerate the model, to score the model, to average	line 4: which k' keep $c > 0$ line 5: $u(c)$ line 6: the sum over z' uses P_z	actions you are only offered rewards you can only observe prices that depend on a distribution
a state space to sweep	line 3: every (k, z)	m^d points with d state variables
nothing to try out	the max inspects every action	samples show only what you tried

- **In two dimensions none of this binds. The requirement that breaks first is not the grid.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “Value function iteration: how DP computes”.

Challenge 1: the Curse of Dimensionality in the State



- ▶ A table with m points per dimension has m^d entries.
- ▶ Go: rules on one page, about 10^{170} positions. A known model, and no table.
- ▶ *Sampling changes which states get visited, not how many numbers a table needs.* Decks 3.5 and 3.6 answer this one.
- ▶ **And the curse has a second source, which appears the moment we improve a learner — Deck 3.2.**

Source: Memory at 8 bytes per entry. Miao & Feng, *AI for Economists*, ch. 11; K. Judd, *Numerical Methods in Economics*, 1998.

Session 3.1

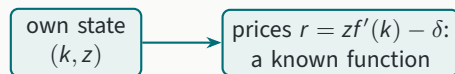
What We Pretend Not to Know

The environment you cannot write down, and the weakest form of not knowing

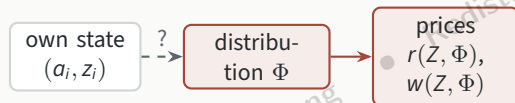


Challenge 2: an Environment You Cannot Write Down

representative agent



heterogeneous agents

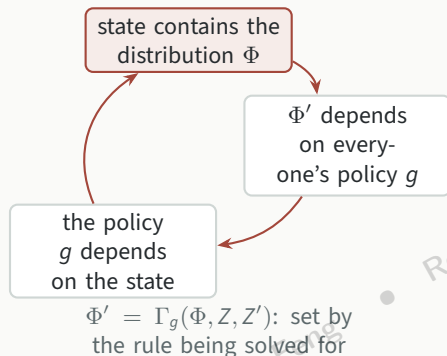


$\Phi' = \Gamma_g(\Phi, Z, Z')$: set by everyone's saving rule

- ▶ Growth model: the household's own state pins down prices.
- ▶ Heterogeneous agents with aggregate shocks: prices depend on the wealth distribution Φ .
- ▶ Its law of motion Γ_g has no closed form in general. It depends on the policy being solved for.
- ▶ **Prices are the environment. Here they must be learned, not specified.**

Source: P. Krusell, A. Smith, *JPE* 106(5), 1998; W. Den Haan, *JEDC* 34(1), 2010; Miao & Feng, ch. 11.

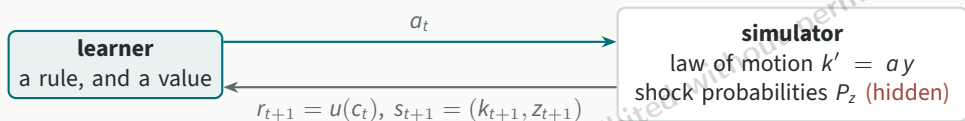
Where the Formula Runs Out — and Two Ways to Write It Anyway



- ▶ Writing Φ into the state makes the problem recursive, and infinite-dimensional.
- ▶ **The law of motion can still be written:** in continuous time by the Kolmogorov forward equation, in discrete time by an analytic, *differentiable* kernel — Deck 3.6.
- ▶ **But an equation for Γ_g is not a table for it:** the state is still a distribution, so challenge 1 remains. Krusell and Smith keep a few moments.
- ▶ **Both challenges at once: a law of motion you must construct, on a state you cannot sweep.**

Source: Aiyagari, *QJE* 109(3), 1994; Krusell & Smith, *JPE* 106(5), 1998; Achdou et al., *REStud* 89(1), 2022. The two-point split is Young, *JEDC* 34(1), 2010; using it as an analytic, differentiable law of motion is Feng, Han, Sargent & Zhu (2026, working paper).

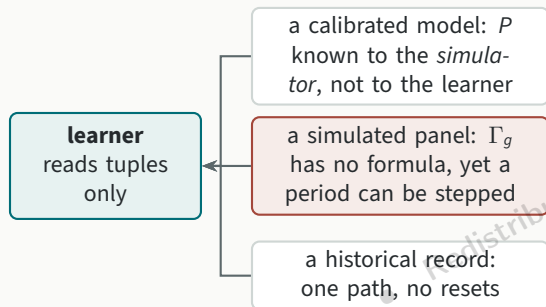
What We Pretend Not to Know: the Shock Probabilities



- ▶ **Hidden from the learner:** the probabilities P_z . Today's z is observed and the simulator hands back z' ; the matrix is never shown, and the lab raises if one is read.
- ▶ **Shown:** the feasible set and a realised reward — not $u(\cdot)$, not f .
- ▶ **Kept by the grader:** the exact V^*, Q^*, g^* . Three words, fixed for the session — that triple is the **answer key**, the loss against it the **grade**, dynamic programming the **benchmark**.
- ▶ **Hiding a P_z that exists is the weakest form of not knowing**, and the book does it on purpose so a learner can be graded state by state. The real case is the previous frame.

Source: Miao & Feng, *AI for Economists*, ch. 11, §11.2.3.

The Answer to Challenge 2: Use Only What Can Be Observed



all three emit the same four numbers

- ▶ A coin's bias is learned by tossing it; a shock process the same way.
- ▶ Nobody can write Γ_g down, yet a simulated panel can be stepped forward.
- ▶ The requirement is weaker than "know the model": one transition at a time.
- ▶ **Every algorithm here reads only realised (s, a, r, s') — state, action, reward, next state. That is what makes challenge 2 passable.**

Source: Krusell & Smith, *JPE* 106(5), 1998; Maliar, Maliar & Winant, *JME* 122, 2021; Miao & Feng, ch. 11.

Dynamic Programming and Reinforcement Learning: the Careful Statement

a sampled Bellman update (Deck 3.4 names it)

$$Q(s, a) \leftarrow Q(s, a) + \eta [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

↓ expectation taken exactly, step size $\eta = 1$

value function iteration $Q(s, a) \leftarrow R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q(s', a')$

- ▶ **Same problem:** both find the fixed point of the same Bellman operator.
- ▶ **Same update, in the limit.** Take the expectation exactly and dynamic programming is what is left — the corner with no approximation of any kind.
- ▶ **What differs is what the update may read** — P , or realised transitions, which is why the sampled form survives challenge 2.
- ▶ **What does not carry over: the guarantees.** Contraction and finite termination belong to the *exact* operator. **Which survive sampling is Deck 3.2's question.**

Source: D. Bertsekas, J. Tsitsiklis, *Neuro-Dynamic Programming*, 1996; J. Tsitsiklis, *Machine Learning* 16, 1994; Sutton & Barto, 2018, ch. 4.

Session 3.1

One Problem, Two Languages

And the growth model, in the two instances this session runs



The Markov Decision Process

- ▶ A **Markov decision process** (MDP) is a five-tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle.$$

- ▶ **States** $\mathcal{S}$; feasible **actions** $\mathcal{A}(s)$; **transition** probabilities $P(s' \mid s, a)$; expected **reward** $R(s, a)$; **discount factor** $\gamma \in [0, 1)$.
- ▶ The **Markov property**: the current state carries everything needed to forecast,

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t).$$

- ▶ Every model in this session is an MDP. What changes from deck to deck is which parts of the five-tuple an algorithm is allowed to read.
- ▶ **From here on γ is the discount factor: the economist's β , nothing else.**

Source: M. Puterman, *Markov Decision Processes*, 1994; Miao & Feng, *AI for Economists*, ch. 11.

When the State Is Not Markov, Enlarge It

- ▶ Whether a problem is Markov depends on what you put in the state: it is a property of the process *and* the state description together, never of the description alone.
- ▶ Habit formation: utility depends on c_{t-1} , so (k, z) no longer forecasts the future. Add c_{t-1} to the state and the property returns.
- ▶ The price is dimension — one axis per remembered variable, which is challenge 1 again. If part of the state is never observed, the problem is a **partially observable** MDP: one acts on a belief.
- ▶ When *no* choice of natural variables works — a non-optimal economy may admit no continuous recursive equilibrium — carry the **continuation values** in the state instead.
- ▶ The Markov property makes a problem *recursive*. It does not make it *computable*.
- ▶ **Choosing the state is a modelling decision. Sometimes the repair is to carry a value as a state — which is exactly what a critic does.**

Source: Z. Feng, J. Miao, A. Peralta-Alva & M. Santos, *International Economic Review* 55(1), 2014; Santos, *JET* (2002), shown in Session 1.4; Kaelbling, Littman & Cassandra, *Artificial Intelligence* 101, 1998.

One Dictionary: the MDP and the Dynamic Economic Model

MDP	Economics	Example
state s	state variables	capital k , wealth, productivity z
action a	control, decision	next capital k' , the policy rate
transition $P(s' s, a)$	law of motion, shocks	$k' = a; \ln z' = \rho \ln z + \eta'$
reward $R(s, a)$	period utility or profit	$u(c)$
discount γ	discount factor β	0.99 quarterly, 0.95 annual
policy $g(a s)$	decision rule	$k' = g(k, z)$
value $V^g(s)$	value function	lifetime utility $V(k, z)$

- ▶ The book writes the policy g ; the RL literature writes π . This session follows the book.
- ▶ **Rule, policy, decision rule** are one object; an **actor** is a parameterised one, g_θ .
Session 2's share s is here the action a , since s is a state.

Source: Miao & Feng, *AI for Economists*, ch. 11, "The MDP and dynamic economic models".

The Same Equation in Two Languages

economics $V(k, z) = \max_{k'} \{u(c) + \beta E[V(k', z') | z]\}, \quad c = zk^\alpha + (1 - \delta)k - k'$



reinforcement learning $V^*(s) = \max_{a \in \mathcal{A}(s)} \{R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')\}$

- ▶ Both say: the value of a state is today's payoff plus the discounted expected value of tomorrow's state.
- ▶ Only the names differ. k is a state, k' is an action, P_z is the transition, β is γ .
- ▶ When the table is too large, both fields reach for function approximation.
- ▶ **The Bellman backbone never changes this session. Only how its expectation is filled.**

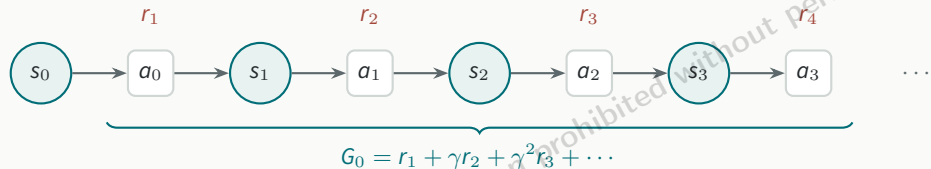
Source: R. Bellman, *Dynamic Programming*, 1957; Miao & Feng, *AI for Economists*, ch. 11.

Policy: a Rule, Deterministic or Random

- ▶ A **policy** maps states to actions. It is the economist's decision rule.
- ▶ **Deterministic**: $a = g(s)$. The optimal saving function $k' = g(k, z)$ is one.
- ▶ **Stochastic**: $a \sim g(\cdot | s)$, a distribution over the feasible actions in each state. **Deck 3.2's rule is the deterministic branch** — a saving rule, as this model writes one. It draws *around* that rule only to find out what else was worth trying.
- ▶ Economists rarely need random rules: with a known model the optimum is deterministic.
- ▶ A learner does need them: samples show only the actions you took, so a rule that never tries the second-best action never learns whether it was second-best.
- ▶ **Randomness in the policy is not noise in behaviour. It is how the data get collected.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “Policy, return and value functions”.

Trajectory and Return



- ▶ A **trajectory** is one simulated path. The reward r_{t+1} is period t 's utility $u(c_t)$; the index says when it is seen.
- ▶ The **return** is the discounted utility stream, $G_t = \sum_{j \geq 0} \gamma^j r_{t+j+1}$.
- ▶ It is recursive: $G_t = r_{t+1} + \gamma G_{t+1}$. Take expectations and the Bellman equation appears.
- ▶ **A return is one random draw of lifetime utility. A value is its expectation.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §3.3; Miao & Feng, *AI for Economists*, ch. 11.

Two Value Functions: V and Q

- ▶ The **state-value function**: expected return from state s when following g ,

$$V^g(s) = E_g[G_t \mid s_t = s].$$

- ▶ The **action-value function**, or **Q function**: take action a first, follow g afterwards,

$$Q^g(s, a) = E_g[G_t \mid s_t = s, a_t = a].$$

- ▶ Linked by $V^g(s) = \sum_a g(a \mid s) Q^g(s, a)$. The **Bellman expectation equation** evaluates a *given* rule, with no max. Call its right-hand side $T^g V$:

$$V^g(s) = R(s, g(s)) + \gamma \sum_{s'} P(s' \mid s, g(s)) V^g(s').$$

- ▶ T^g is a contraction of modulus γ , exactly as T is. The max is what made T hard to apply without the model; T^g has no max — and Deck 3.2 shows that this is precisely the guarantee that survives when the model goes.

Source: Miao & Feng, *AI for Economists*, ch. 11, “The recursive form of the principle of optimality”.

Why a Learner Keeps Q — and What Happens When It Cannot

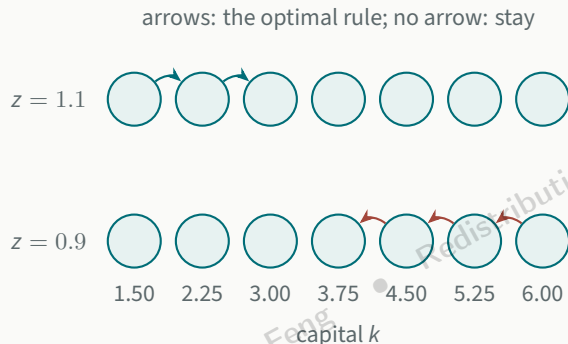
	next capital k'				
	1.50	2.25	3.00	3.75	4.50
$k = 3.75$	1.88	2.90	3.51	3.74	3.05

one row of Q : the largest entry is the improved rule

- ▶ With V^g : comparing actions needs $R + \gamma \sum_{s'} P V^g$. **The model is back.**
- ▶ With Q^g : one row, the expectation already inside the samples — **that is how a learner improves a rule without a model.**
- ▶ **But that is the grid.** On the continuous model k and the share are both real: no cell, no finite row. A max is still available over a fine grid of shares — as the key takes — but only with the model, or a learned $\hat{Q}$ (Deck 3.5).
- ▶ **A gradient needs neither, and it survives Session 4.**

Source: Exact Q^* at $(k, z) = (3.75, 0.9)$, 7×2 grid. Miao & Feng, ch. 11.

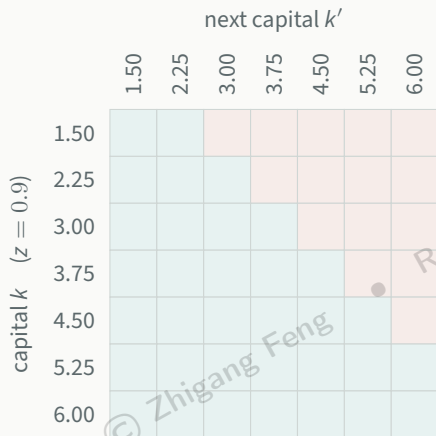
Two Instances of One Model



- ▶ **The 7×2 grid** (drawn): 14 states, the action is next capital k' ; every value can be printed. **Decks 3.3–3.4 run here.**
- ▶ **The continuous model** (frame 3): k real, the action the share $a \in (0, 1)$, the key a 400-point DP. **Deck 3.2 runs here.**
- ▶ **The action is not the same object in the two:** on the grid it is k' , here the share. “Equivalently” holds for the model, not for the action space.

Source: Miao & Feng, ch. 11, “The benchmark model”; rule from the Session 3 lab, block A.

Feasible Actions and the Action Mask



Source: Session 3 lab, block A; 7×2 grid, $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2.

- ▶ Consumption must be positive, so the feasible set $\mathcal{A}(k, z)$ depends on the state.
- ▶ Teal cells are feasible: 34 at each z , 68 pairs in all.
- ▶ An **action mask** enforces it: infeasible entries get $-\infty$ before any $\arg \max$.
- ▶ **The continuous instance needs no mask at all:** $k' = ay$ with $a \in (0, 1)$ keeps consumption positive by construction. That is Session 2's sigmoid share, and it is why Deck 3.2 never checks feasibility at all.

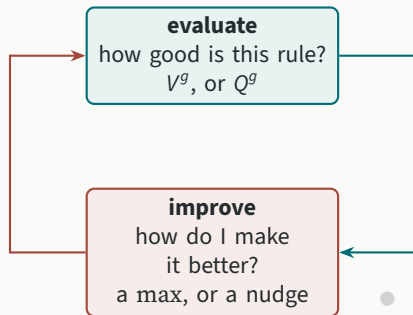
The Environment as Code: `reset` and `step`

```
env = make_env_7x2(seed=0)
state = env.reset(k_idx=3, z_idx=1)      # start at k = 3.75, z = 1.1
mask = env.action_mask(state)           # which k' leave consumption positive
state, reward, terminated, truncated, info = env.step(3)      # choose k' = 3.75
# -> state (3, 1) or (3, 0), reward = ln(1.3953) = 0.3331, terminated = False
```

- ▶ `step` draws tomorrow's productivity inside the simulator. The learner gets the draw, never the probabilities.
- ▶ `terminated` is always false (infinite horizon); `truncated` ends a simulated segment. Treating truncation as the end of time would value all later saving at zero.
- ▶ The continuous instance has *the same two methods*: `reset` puts you somewhere in the band, `step` takes a share and returns r and s' .
- ▶ **This interface is all any algorithm in the lab may touch.**

Source: Book code, ch. 11 (the model) and ch. 12 (the interface), as vendored in the Session 3 lab, block A.

Two Questions Alternate All Session



every method here is a choice of how to answer each, and how far to run one before switching

- ▶ Run *evaluate* to convergence and then improve, and you have **policy iteration**. Take one backup instead and you have value function iteration, which Session 1.4 gave you.
- ▶ Policy iteration appears nowhere in Sessions 1 or 2. Deck 3.2 defines it in one sentence, because its loop is the one every learner in this session runs.
- ▶ **Deck 3.2 answers both at once — on the continuous model, with P_z hidden, and with eight numbers.**

Source: Miao & Feng, *AI for Economists*, ch. 11 — the two alternating questions; R. Howard, *Dynamic Programming and Markov Processes*, 1960.

Session 3.2

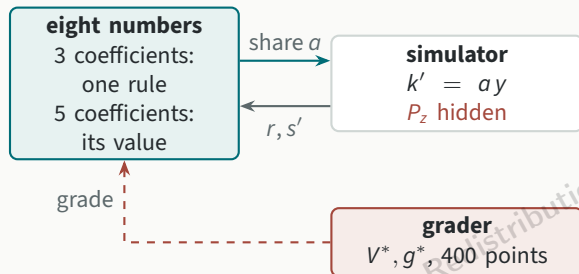
The Simplest Actor–Critic: How It Works, Why It Works, and the Wall It Meets

Nine numbers on the continuous growth model, with the shock probabilities hidden



Zhigang Feng

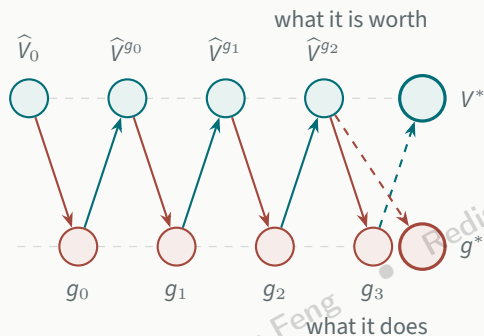
The Question This Deck Answers



- ▶ Capital is a real number and P_z is hidden. **Can eight numbers find the rule dynamic programming finds — on a 400-point grid, as the answer key?**
- ▶ **Hidden:** P_z . **Shown:** a realised reward and next state. **Kept by the grader:** V^*, g^* .
- ▶ No table, no grid, no network: **one saving rule with three coefficients, and one value function with five.**

Source: The model of Deck 3.1, frame 3; $\alpha = 0.36, \beta = 0.95, \delta = 0.10, u = \ln c, z \in \{0.9, 1.1\}$ with switching probability 0.2. Answer key and learner: Session 3 lab, block A2.

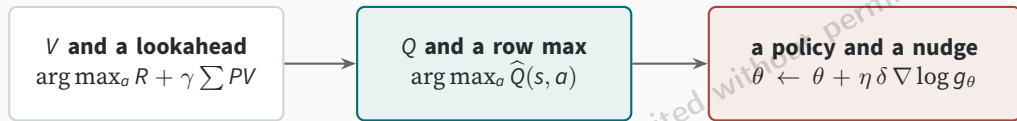
Evaluate, Then Improve: the Loop Every Method Here Runs



- **Generalized policy iteration:** alternate *evaluation* (teal) and *improvement* (red).
- **Howard's policy iteration** evaluates a rule *to convergence* before improving it; value iteration takes one backup instead.
- **This deck takes one *sampled* backup and one nudge, every transition — three points on one dial.**

Source: R. Howard, *Dynamic Programming and Markov Processes*, 1960; Sutton & Barto, 2018, §4.6; Miao & Feng, ch. 11.

Improving a Rule Means Comparing Actions — and a Third Road That Does Not



needs P

needs a max

needs neither

dynamic programming

SARSA, Q-learning

actor-critic

- ▶ **The left two are the 7×2 grid's.** With V you must look one step ahead, which needs P . With Q the comparison is already inside the table: **read the row for the state you are in, take its largest entry** — no model, no derivative, just a comparison.
- ▶ **The right one is this deck's.** On the continuous model k is real, so there is no cell to hold Q ; the share is real, so there is no finite row to scan.
- ▶ A max is still available over a fine grid of shares, as the key takes — but only with the model, or a learned $\hat{Q}$ (Deck 3.5).
- ▶ **A gradient needs neither, and it survives Session 4.**

Source: Miao & Feng, ch. 12, “From value-based to policy-based methods”. Deck 3.1 draws the row.

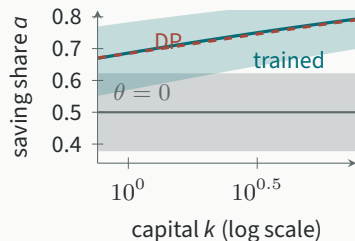
Session 3.2

The Simplest Actor-Critic

Four numbers choose, five numbers judge, one number moves both



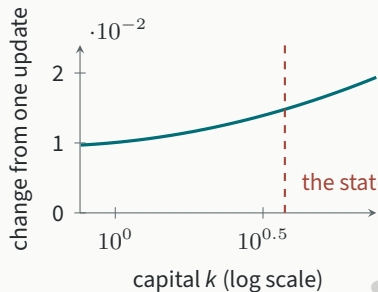
The Actor: a Saving Rule, Three Numbers



- ▶ **The rule is a deterministic function of the state**, as the growth model writes one:
 $a = \text{sigmoid}(\theta_0 + \theta_1 k + \theta_2 \mathbf{1}\{z = 1.1\})$. **Three numbers.**
- ▶ With no model you cannot learn what a *different* share is worth without trying one, so while training we **probe**: draw around the rule at a fixed width — the bands. **The probe is machinery, not the rule**: not learned, switched off to grade.
- ▶ It starts flat at 0.5000 and ends on DP's curve. **The band never narrows; what moves is the line.**

Source: Session 3 lab, block A2; $z = 0.9$, untrained and at 10^6 steps. Bands: one probe width either side.

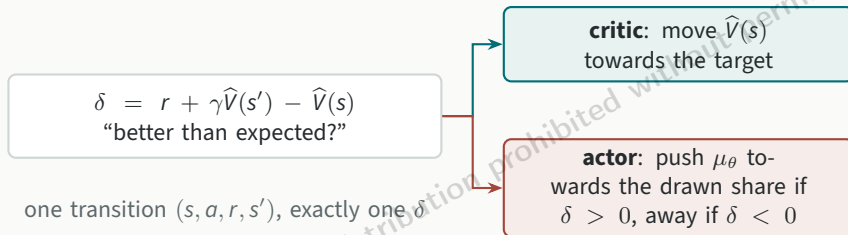
The Critic: Five Numbers, and What One Update Does



- ▶ $\hat{V}_\psi = \psi^\top \phi$, with $\phi = (1, \tilde{k}, \tilde{k}^2, \mathbf{1}\{z = 1.1\}, \tilde{k} \mathbf{1}\{z = 1.1\})$.
- ▶ It starts flat at $u(c^*)/(1 - \gamma) = 4.27$, while the true V^* runs from -0.25 to 6.48 across the band.
- ▶ The plot is what *one* update did. The transition happened at $k = 3.75$, and **the value moved at every other k too** — 0.010 at the bottom of the band, 0.019 at the top. **There is no cell to correct.**
- ▶ The target $r + \gamma \hat{V}_\psi(s')$ uses the critic's own estimate: **bootstrapping**.

Source: Session 3 lab, block A2; the first update of the worked period on frame 8.

The TD Error: One Number, Two Jobs



- ▶ δ compares what happened with what the critic expected. It estimates the **advantage** $Q^g(s, a) - V^g(s)$: a grade for the action.
- ▶ Session 2 warned that sampling an *inner* expectation is not harmless — it sat inside a logarithm and a square. **Here the update is linear in the target, so a draw is unbiased.**
- ▶ **Evaluation and improvement from one number — frame 2's loop at full speed.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "Actor-critic"; Sutton & Barto, 2018, §13.5.

The Whole Algorithm on One Page

One-step actor-critic, in the book's notation

- 1 initialise θ (three numbers), ψ (five); fix the probe width σ ; draw s from the band
 - 2 **for** $t = 0, 1, 2, \dots$:
 - 3 probe the rule: $x \sim N(\mu_\theta(s), \sigma^2)$, $a = \text{sigmoid}(x)$; the simulator returns r and s'
 - 4 $\delta \leftarrow r + \gamma \hat{V}_\psi(s') - \hat{V}_\psi(s)$ *at a reset, bootstrap as usual*
 - 5 $\psi \leftarrow \psi + \eta_\psi \delta \phi(s)$ *critic: one step*
 - 6 $\theta \leftarrow \theta + \eta_\theta \delta \nabla_\theta \ln g_\theta(a | s)$ *actor: one step*
 - 7 every 300 periods redraw s from the band; otherwise $s \leftarrow s'$
- The **score** closes in one expression, $\nabla_\theta \ln g_\theta(a | s) = \frac{x - \mu}{\sigma^2} (1, \tilde{k}, \mathbf{1}\{z = 1.1\})$. The sigmoid's Jacobian does not depend on θ , and σ is not a parameter, so nothing else appears.
 - **Line 4 is the only place the future enters, and it enters through the critic.**

Source: Miao & Feng, ch. 12, §12.5.1, “one-step actor-critic”; the comment on line 4 is the book's.

A Worked Period: One Transition, Every Number

```
state      k = 3.7500  z = 0.90      ktilde = +0.3814
actor      mu = 0.0000  sigma = 0.5000  eps = +0.1257  ->  x = +0.0629
action     a = sigmoid(x) = 0.5157      (DP's share here: 0.7549)
model      y = 4.8234  k' = a y = 2.4875  c = (1-a) y = 2.3359
reward     r = ln c = +0.8484            next shock z' = 0.90
delta      r + gamma V(s') - V(s) = +0.6347  (V(s) = V(s')) = 4.2732
```

```
psi  4.2732 0 0 0 0  ->  4.2859  0.0048  0.0018  0  0
theta 0 0 0          ->  0.000032  0.000012  0      (probe fixed at 0.5000)
```

- ▶ The rule drew 0.516 where DP saves 0.755, so it under-saved; the reward was still good, the critic had expected less, and $\delta > 0$ pushed μ_θ towards what was drawn.
- ▶ **The critic's constant moved 0.0127; the actor's moved 0.000032 — four hundred times slower.** That is the two-time-scale design: the critic must know what a state is worth before the actor is allowed to believe it.

Source: Written by the lab so the slide cannot drift, seed 0, untrained start; Session 3 lab, block A2.

Why No Transition Matrix Appears Anywhere

a path's probability $\Pr(\tau) = \underbrace{\mu(s_0) \prod_t P(s_{t+1} | s_t, a_t)}_{\text{the environment}} \cdot \underbrace{\prod_t g_\theta(a_t | s_t)}_{\text{the rule}}$

the environment's terms do not contain θ

take logs, differentiate in θ $\nabla_\theta \log \Pr(\tau) = \sum_t \nabla_\theta \log g_\theta(a_t | s_t)$

- ▶ P enters the probability of a path but not its *derivative*: it differentiates away.
- ▶ The critic needs no P either — its target $r + \gamma \hat{V}(s')$ is built from one realised draw.
- ▶ **This is why the method survives challenge 2. It is also Deck 3.5's policy-gradient theorem, one slide early.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “The policy gradient theorem”; Sutton & Barto, 2018, §13.2.

Session 3.2

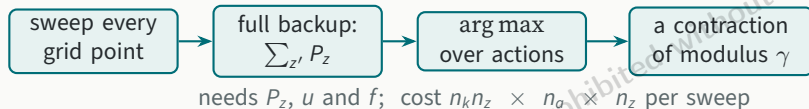
Why It Works, and What Survives

The contraction is still there — for a fixed rule

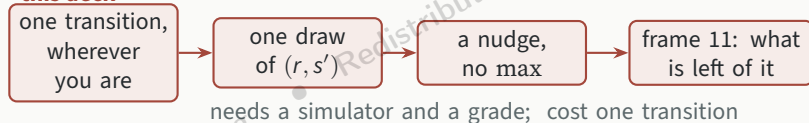


Compared with Dynamic Programming, Drawn

dynamic programming



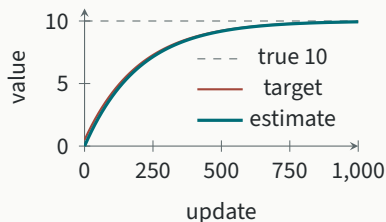
this deck



- ▶ Every column is a trade: the sweep for a visit, the sum for a draw, the maximum for a nudge.
- ▶ **The first three make challenge 2 passable. The fourth is the bill.**

Source: Miao & Feng, ch. 11 and ch. 12, §12.6.1.

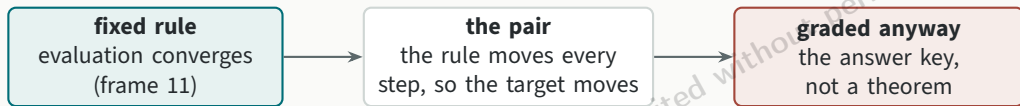
What Survives: the Contraction, for a Fixed Rule



- ▶ Hold the rule fixed. The expected target is $T^g \hat{V}$ — Deck 3.1's operator, a contraction of modulus γ , and **sampling applies it on average**.
- ▶ With *this* critic it reaches the **projected fixed point**: the best linear fit to $T^g \hat{V}$ over the states the rule visits, within $1/(1 - \gamma) = 20$ of that fit — or ≈ 3.2 by the book's tighter argument.
- ▶ The binding condition is that the data come from the rule being evaluated. Ours do, up to a reset every 300 periods, and $\gamma^{300} \approx 2 \times 10^{-7}$.
- ▶ **The max threw the contraction away. A fixed rule gets it back.**

Source: Tsitsiklis 1994 and Jaakkola, Jordan & Singh 1994 (tabular); Tsitsiklis & Van Roy 1997 (linear) — the book's theorems 12.1 and 12.2.

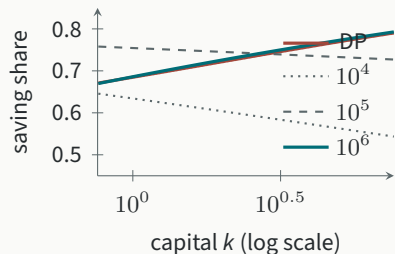
What Does Not Survive, and What Replaces It



- ▶ Once the actor moves, the critic chases a moving target and Howard's finite termination is gone.
- ▶ What is known covers neighbouring cases — exact gradients on softmax rules, two time scales with a linear critic — **under conditions this run does not meet**, which is what the book says of its own chapter's algorithms.
- ▶ **So the guarantee this session runs on is the answer key.** Deck 3.5 frame 18 states the conditions and closes the debt.

Source: Konda & Tsitsiklis, *NeurIPS* 12, 2000; Agarwal, Kakade, Lee & Mahajan, *JMLR* 22, 2021; Mei et al., *ICML*, 2020. Miao & Feng, ch. 12, §12.5.1 footnote.

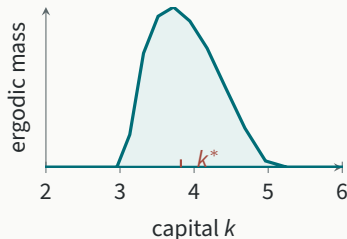
Measured: the Rule It Learned, against the Rule DP Found



- ▶ $z = 0.9$; the $z = 1.1$ panel is the same picture — the learned shift is -0.001 .
- ▶ By 10^6 steps the rule sits on DP's curve, and **it finds the slope, not just the level:** $\theta_1 = 0.317$ against the 0.320 that is the best this shape can do.
- ▶ **That best rule scores** 0.00001 %, so everything on the next frame is *learning* error.
- ▶ **The critic is not accurate** — rms 1.72 against a V^* running -0.25 to 6.48 . It does not need to be: **it only has to rank actions.**

Source: Session 3 lab, block A2; median of 20 seeds. Continuous model.

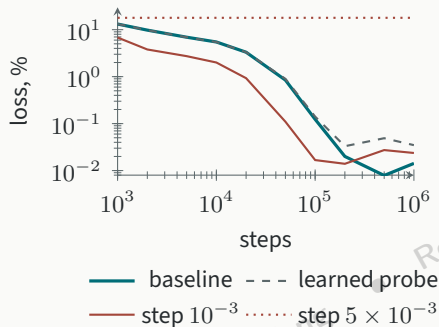
Where the Economy Lives, and the Grade



- ▶ A rule is graded where its own economy goes: here between $k = 3.0$ and 4.8 , though the band runs to 7.6 .
- ▶ **Median of 20 seeds: 0.014 % where the economy lives, 0.015 % over the band; best seed 0.002 %, worst 0.106 %.**
- ▶ A **consumption-equivalent welfare loss**: the fraction of consumption given up, for ever, to hold the learned rule instead of DP's.
- ▶ **Session 2 used those words for Judd's Euler-residual scale — a different object.**

Source: Session 3 lab, block A2; the probe is off when the rule is graded, and the density comes from Young's two-point split.

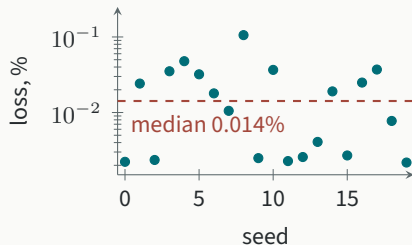
Three Knobs Decided the Answer



- ▶ **Actor step.** 2×10^{-4} gives 0.014 %; 10^{-3} , 0.024 %; 5×10^{-3} destroys the run.
- ▶ **Probe width.** 0.5 is tuned: at 0.25 the rule finds the level but not the slope ($\theta_1 = 0.17$ against 0.32); at 1.0 the run is destroyed.
- ▶ **Learning the width instead costs 0.014 %** → **0.035 %** — σ falls to 0.13 and the rule stops trying alternatives. **That is why the probe is a knob, not a parameter.**
- ▶ **Resets.** 0.014 % with, 0.018 % without — *exploring starts by another name.*

Source: Session 3 lab, block A2; medians of 20 seeds at 10^6 steps.

When It Fails, How It Fails



- ▶ Twenty seeds, same settings: 0.002 % to 0.106 %. **One run is not a measurement.**
- ▶ **Every one of the twenty over-saves**, and the spread is the *level*, not the shape: the error in θ_0 tracks the loss almost exactly, the error in θ_1 not at all.
- ▶ Under- and over-saving of the same size cost the same, so **the bias is in the estimator, not the objective.**
- ▶ The first place to look is the γ^t weight the theorem asks for and this demo drops — **Deck 3.5 puts a price on exactly that.**

Source: Session 3 lab, block A2; 20 seeds, 10^6 steps. Weighted share error against DP's rule.

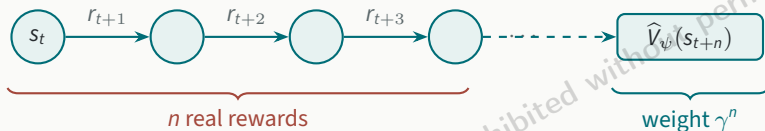
Session 3.2

Improve It: Roll Out T Periods

And the wall that puts in the way — uncertainty, unfolded



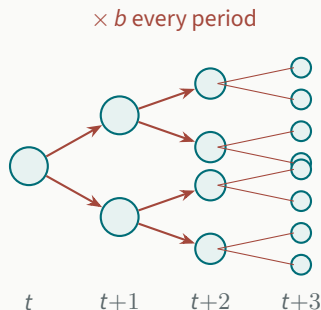
Why Roll Out at All



- ▶ Both halves of the update lean on the critic. With $n = 1$ it carries weight $\gamma = 0.95$; take n real rewards first and its weight falls to γ^n .
- ▶ The **n -step return** is $\hat{G}_{t:t+n} = \sum_{j < n} \gamma^j r_{t+j+1} + \gamma^n \hat{V}_\psi(s_{t+n})$, and the advantage becomes $\hat{G}_{t:t+n} - \hat{V}_\psi(s_t)$.
- ▶ In dynamic programming this is **modified policy iteration**. It is not Howard's improvement theorem.
- ▶ **Fewer numbers taken on trust. The next two frames are what it costs.**

Source: Miao & Feng, ch. 11, §11.4.2, eq. (11.23) and its footnote. The usual attribution for modified policy iteration is Puterman & Shin, *Management Science* 24, 1978.

The Unfolding of Uncertainty

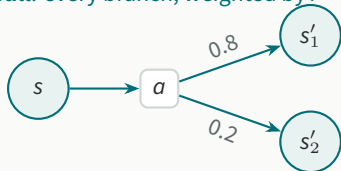


- ▶ On a table the rollout is a recursion: histories re-merge on the grid. **The key's grid re-merges; the learner's path does not.**
- ▶ From one state, T periods ahead is a tree of b^T shock histories: $2^{20} \approx 10^6$ here, $7^{20} \approx 8 \times 10^{16}$ with seven shock nodes.
- ▶ **One more state variable multiplies the table by m ; one more period multiplies the tree by the branching factor b .**
- ▶ **A second curse, in time — created by the answer to challenge 2.**

Source: Miao & Feng, ch. 11, §11.4.2.

Sampling the Tree

full: every branch, weighted by P



sample: one branch, no probabilities

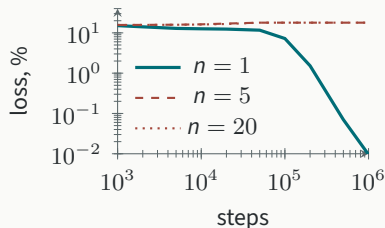


Source: Miao & Feng, ch. 11, §11.4.3.

	full	sample
reads	P	realised (r, s')
T steps ahead	b^T	$N \times T$
error	none	$\sigma_G / \sqrt{N}$
grows with d, b ?	yes	no

- ▶ A **sample backup** replaces the sum by walks: N of length T , cost $N \cdot T$, error $\sigma_G / \sqrt{N}$ — free of both b and d .
- ▶ Large T is **Monte Carlo** (3.3); $T = 1$ is **temporal difference** (3.4).
- ▶ **The rest of the session is the choice of T , and of what to store.**

Measured: Rolling Out Made It Worse



- ▶ Four actor steps tried per n , each curve that n 's best. **Only $n = 1$ learns;** $n = 5$ and $n = 20$ end at the band's floor at every step tried.
- ▶ Not the code — with the actor fixed, n -step evaluation reaches the same critic for every n ; nor the critic's step, which can move tenfold and change nothing.
- ▶ **It is the rule's own path:** with $n = 20$, twenty periods of the *current* rule run before the critic is consulted, and early on that rule wastes capital.
- ▶ **Not bias against variance, but the bias a bad rule puts into its own long return.**

Source: Session 3 lab, block A2; medians of 20 seeds. A collapsed run grades 17.9 %: the rule pinned at the band's floor.

Session 3.2

Why This Is Not Yet Enough

What eight numbers bought, and what they did not



The Other Road, on a Grid



- ▶ On the 7×2 grid the maximum is cheap and it is the better road: **Deck 3.4 introduces SARSA and Q-learning**, and Deck 3.6 puts them on the same grade as this deck.
- ▶ This deck could not take that road, because a table needs cells and this model has none — which is the honest reason, not a preference.
- ▶ **The two roads swap places in Session 4.** There the action is a whole saving function; there is no row, and no grid over it either.

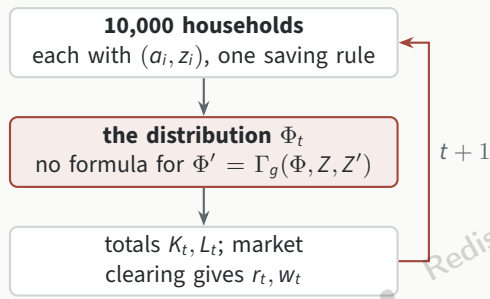
Source: Miao & Feng, *AI for Economists*, ch. 12, §§12.2 and 12.4.

What the Actor–Critic Bought, and What It Did Not

against dynamic programming	where it is paid for
✓ reads samples, never P_z	challenge 2, frame 9
✓ no maximum over actions	frame 3
✓ no grid over the state — eight numbers	frame 5
× the rule and the value are only as good as their <i>shape</i>	Deck 3.5
× a rollout unfolds uncertainty	frames 18–20, Decks 3.3–3.4
× guarantees: evaluation yes, improvement conditional	frame 12, Deck 3.5

- ▶ Frame 10 drew the mechanics. **This is the bill.**
- ▶ **Three ticks, three crosses, and the crosses are the rest of the session.**

The Model Economists Actually Want



- ▶ Krusell–Smith: the state becomes $(a_i, z_i; Z, \Phi)$, and Φ as a histogram is n_b numbers, not two.
- ▶ **A rule linear in five features cannot carry that** — the features must be learned (3.5).
- ▶ Φ 's law of motion must come from somewhere: the KFE in continuous time, an analytic differentiable kernel in discrete time, or a simulated panel. **Deck 3.6 builds one.**
- ▶ **Nine numbers were never the point. The loop was.**

Source: Krusell & Smith, *JPE* 106(5), 1998; Miao & Feng, ch. 12, §12.7.

Nine Numbers, Six Ideas, and Where Each Is Earned

in the demo	the idea, used on credit	earned in
one draw of (r, s') per update	sample the expectation	3.3, frames 1–3
the target $r + \gamma \hat{V}_\psi(s')$	bootstrap one step; how far to roll out	3.4, frames 3–6
$\hat{V}_\psi = \psi^\top \phi$, five numbers	approximate the value: features, then networks	3.5, route one and two
a saving rule, three numbers	improve by a gradient, not a max	3.5, policy gradient
both moved by one δ	evaluation inside improvement	3.5, actor–critic
a fine-grid DP as the grade	an answer key before any learner runs	3.1, then 3.6

- ▶ **Every row is something this deck used without earning.** Each of the next four decks opens by naming the row it pays for — and **Deck 3.6 closes this table.**

The Session's Map, with This Deck Lit



- ▶ Axes: value-centred → policy-centred, tables → parameters. **Every arrow is one deck of this session, and its label is what that step changed.**
- ▶ **This deck sits at the far corner, and got there in one jump.**

Source: After Miao & Feng, ch. 12, figure 12.1; the arrow labels are the book's.

Session 3.3

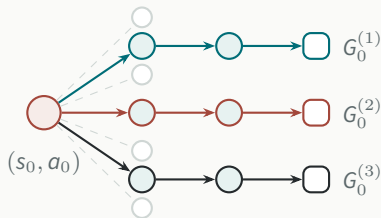
Monte Carlo: Evaluation, Control, and the Data

Average simulated returns to value a rule; then find out what improving a rule from its own data costs



Zhigang Feng

Basic Monte Carlo: Walk N Paths from One Cell, Average the Returns



Basic Monte Carlo for one cell

- 1 **input:** $g, (s, a), N$ paths, horizon T
- 2 **for** $i = 1, \dots, N$:
- 3 simulate T periods: $a_0 = a$, then g
- 4 $G_0^{(i)} \leftarrow \sum_{j=0}^{T-1} \gamma^j r_{j+1}$
- 5 **return** $\hat{Q}^g(s, a) \leftarrow \frac{1}{N} \sum_i G_0^{(i)}$

- ▶ Deck 3.2 ran the one-step end of the dial on a continuous model. This deck starts at the other end — the simplest estimator, on the smallest instance.
- ▶ **Monte Carlo** (MC): average the returns of N simulated paths. Paths are truncated at $T = 300$, and with returns started in the first 150 periods the omitted tail is at most $\gamma^{150} \max |r| \approx 0.003$.
- ▶ **Cost per cell** $O(NT)$, whatever the number of states or shock histories.

Source: Miao & Feng, ch. 11, §11.4.3 — the algorithm “basic Monte Carlo: estimating Q for a single (s, a) ”.

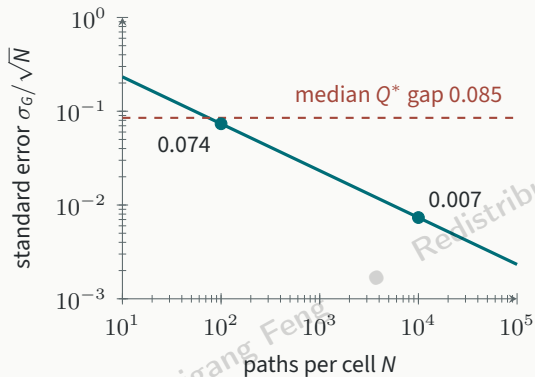
The Exact Q^* Table: the Answer Key for Decks 3.3 and 3.4

$k \setminus k'$	1.50	2.25	3.00	3.75	4.50	5.25	6.00
1.50	0.563	-0.004	-	-	-	-	-
2.25	1.226	1.932	1.441	-	-	-	-
3.00	1.609	2.533	2.946	2.414	-	-	-
3.75	1.879	2.898	3.511	3.735	3.054	-	-
4.50	2.088	3.160	3.864	4.277	4.275	3.620	-
5.25	2.259	3.366	4.122	4.623	4.803	4.784	4.126
6.00	2.403	3.534	4.325	4.877	5.144	5.303	5.257

- ▶ The $z = 0.9$ half of the 7×2 grid. Bold: row maximum, the optimal rule. Best against second best: 0.224 at $k = 3.75$, 0.002 at $k = 4.50$.
- ▶ **A learner must estimate Q more finely than these gaps. This table is the answer key for this deck and for Deck 3.4.**

Source: Miao & Feng, *AI for Economists*, ch. 11, recorded by the book's script; reproduced in the lab, block A. 7×2 grid, $\beta = 0.95$.

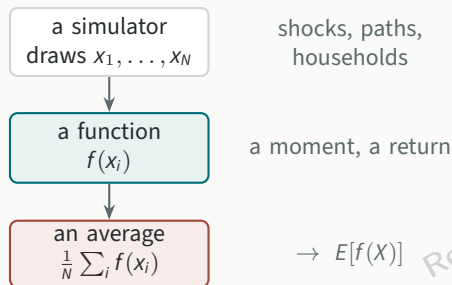
Why a Sample Mean Can Replace the Expectation



- ▶ Law of large numbers: $\hat{Q} \rightarrow Q^g$.
Central limit theorem: the error is about $\sigma_G / \sqrt{N}$.
- ▶ Here $\sigma_G \approx 0.74$: telling best from second best at 95% confidence takes about 400 paths a cell.
- ▶ Ten times the precision costs a hundred times the paths.
- ▶ **The rate depends on neither the dimension of the state nor the number of histories.**

Source: Recorded by the book's script: $\bar{s} = (3.75, 0.9)$, optimal rule, 20,000 paths; 7×2 model, $\beta = 0.95$.

What Monte Carlo Is, in General



- ▶ Monte Carlo estimates *any* expectation by averaging draws, $E[f(X)] \approx \frac{1}{N} \sum_i f(x_i)$. None of the three ingredients is P .
- ▶ **Economists already use this:** the simulated method of moments and indirect inference are the same estimator with a different f .
- ▶ Here f is a discounted return, and the average improves a *rule*, not a parameter.
- ▶ **The estimator is old. What is new is what gets averaged, and what for.**

Source: D. McFadden, *Econometrica* 57(5), 1989; Gouriéroux, Monfort & Renault, *Journal of Applied Econometrics* 8, 1993; Miao & Feng, ch. 11.

Dynamic Programming or Monte Carlo: When to Use Which

	Dynamic programming	Monte Carlo
needs updates error	the law of motion and P every state, every sweep none from sampling	a simulator that produces paths visited cells, after a path ends sampling variance, falling like $1/\sqrt{N}$
cost in dimension d	grows as m^d	does not grow

- ▶ **Stay with dynamic programming** when a state is small enough to sweep, P can be written down, and the error bound is worth having. **Leave it when any one fails.**
- ▶ “Converged” then means two things: an average settling, or a fixed point that stops moving.
- ▶ **Model-free is not error-free: a misspecified simulator passes its error straight on.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “DP and MC compared”.

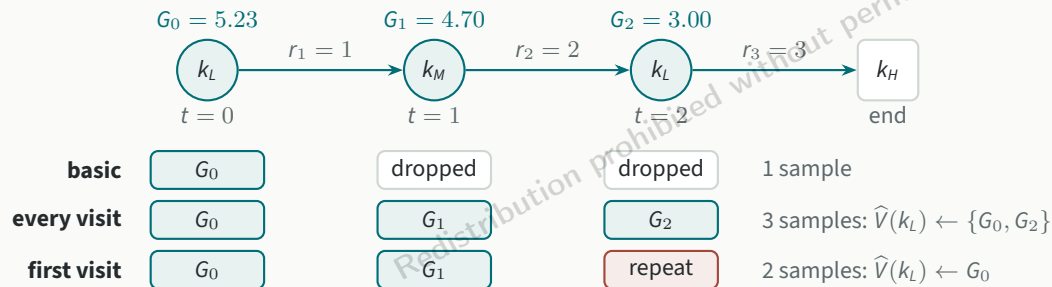
Session 3.3

One Path, Many Samples

Reuse every visit a path makes, and pay for it in coverage



One Path Can Update Every Cell It Visits



- ▶ $\gamma = 0.9$, so $G_0 = 5.23$; by the Markov property G_2 is a sample of $V(k_L)$ too.
- ▶ With $C(s)$ the **visit count: every-visit** counts each visit and is biased of order $1/C(s)$ with lower variance; **first-visit** counts only the first and is unbiased.
- ▶ **This session uses first-visit, so the $1/\sqrt{N}$ rate applies as it stands.**

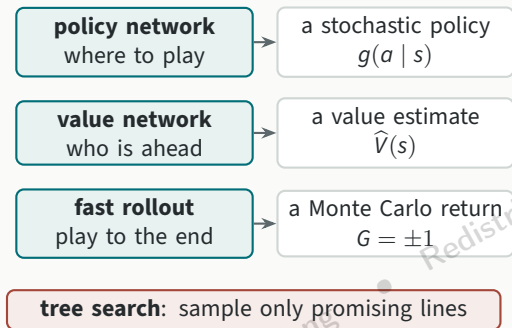
Source: Miao & Feng, *AI for Economists*, ch. 11, "First visit and every visit".

The Price of Reusing Paths: Coverage Becomes the Policy's Job

- ▶ Basic Monte Carlo names every cell as a starting point. Coverage holds by construction, and most of each path is thrown away.
- ▶ Path reuse keeps all of it. But now a cell gets data only if the policy walks there.
- ▶ To evaluate one fixed rule this is harmless: the cells the rule never reaches do not matter for its value where it lives.
- ▶ To *improve* a rule it is decisive: nothing is learned about actions the rule never takes. The second half of this deck is about that.
- ▶ A second limit: G_t contains a long stretch of future shocks, so its variance is high, and nothing can be updated until the path ends.
- ▶ **Low bias, high variance, late updates. Deck 3.4 changes all three at once.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “The trade-off between first visit and every visit”.

AlphaGo: a Policy, a Value, and Monte Carlo Returns



- ▶ Go: about 2.1×10^{170} legal positions, some 250 moves per turn. Ten moves ahead is $250^{10} \approx 10^{24}$ lines.
- ▶ The rules are known. A full backup is still impossible.
- ▶ Policy network: trained on 30 million human positions, then improved by self-play.
- ▶ **Every component is an object from this session. None enumerates the positions.**

Source: D. Silver et al., "Mastering the game of Go with deep neural networks and tree search", *Nature* 529, 2016.

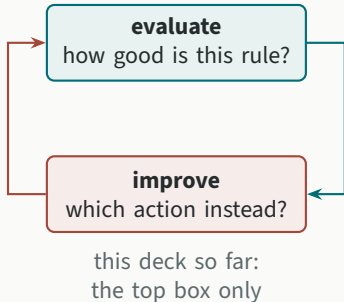
Session 3.3

From Evaluation to Control

Improve the rule from its estimated values, and find out what estimation breaks



Evaluation Was the Easy Half. Now Improve



- ▶ Everything so far values a *fixed* rule. Nothing has tried to change one.
- ▶ Deck 3.2 ran both halves together, one step at a time. Here they are separated, because they fail differently.
- ▶ Wrapping evaluation in an improvement loop is **control**. A warning: in Session 1.4 the control was the choice variable k' — the word's other standard meaning.
- ▶ **The next three slides are what goes wrong when an $\arg \max$ is fed an estimate.**

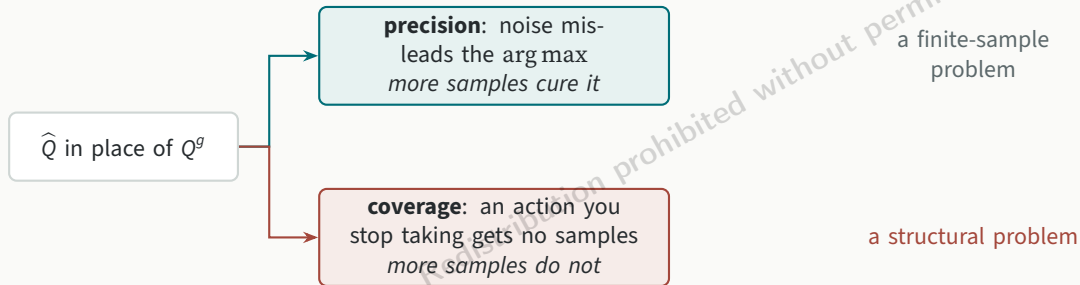
Source: R. Howard, *Dynamic Programming and Markov Processes*, 1960; Miao & Feng, *AI for Economists*, ch. 11.

The Policy Improvement Theorem

- ▶ **Theorem.** In a finite MDP with finite rewards, let g and g' be deterministic. If $Q^g(s, g'(s)) \geq V^g(s)$ at every s , then $V^{g'}(s) \geq V^g(s)$ at every s .
- ▶ The greedy rule $g'(s) \in \arg \max_a Q^g(s, a)$ satisfies the condition. If it brings no gain, g was already optimal.
- ▶ Reading: switching to a better action today only, and carrying on as before, cannot hurt. Do it every day and the whole path is switched.
- ▶ Why: $V^g \leq T^{g'} V^g$; the operator $T^{g'}$ is monotone and a contraction, so iterating it climbs to $V^{g'}$.
- ▶ A finite MDP has finitely many deterministic rules, so exact policy iteration stops at the optimum in finitely many steps.
- ▶ **The theorem assumes the exact Q^g . Monte Carlo hands us $\hat{Q}$.**

Source: R. Howard, *Dynamic Programming and Markov Processes*, 1960; Miao & Feng, *AI for Economists*, ch. 11.

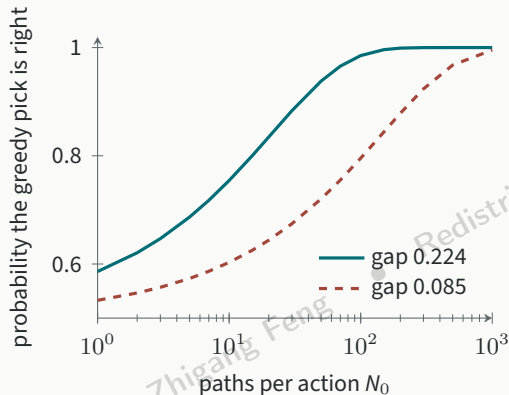
Estimation Breaks the Improvement Theorem — Two Ways



- ▶ The theorem promised: greedy on the *exact* Q^g is never worse. With $\hat{Q}$ it fails in two ways, needing different cures.
- ▶ **One creates mistakes. The other makes them permanent.** Deck 3.2 shows the second on a model with no cells at all: its bad seeds all miss in the same direction, and no amount of data inside the region they visit tells them so.

Source: Miao & Feng, *AI for Economists*, ch. 11, “MC control: where the coverage problem comes from”.

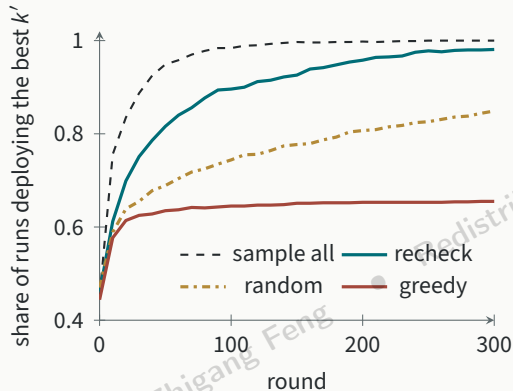
(1) Precision: When Noise Wins the $\arg \max$



- ▶ Two actions, gap Δ , N_0 paths each. The greedy pick is wrong with probability $\Pr\{N(0, 1) > \Delta/(\sigma_G \sqrt{2/N_0})\}$.
- ▶ At $\bar{s}$: right 0.586 of the time at $N_0 = 1$, 0.938 at $N_0 = 50$.
- ▶ Evaluation error $\bar{e}$ can cost $2\bar{e}/(1 - \gamma)^2$ in value: $800 \bar{e}$ here.
- ▶ **Small gaps and few samples: the $\arg \max$ picks noise.**

Source: Normal approximation, $\sigma_G = 0.728$ (book's script). Bertsekas & Tsitsiklis, *Neuro-Dynamic Programming*, 1996.

(2) Coverage: the Actions You Stop Taking



- ▶ One cell, five actions, one path each to start. Each round: deploy the greedy pick, add samples.
- ▶ Sample every action and you reach 0.98 by round 100: there, precision was the whole problem.
- ▶ Pure greedy stalls at 0.66 *for ever*: an unlucky best action is never sampled again, so its error never shrinks.
- ▶ **Rechecking every tenth round costs 0.4 samples a round and reaches 0.98.**

Source: Recorded by the book's script: 2,000 repetitions at $\bar{s}$; "random" tries a random action 10% of the time.

Exploring Starts

Monte Carlo control with exploring starts (first-visit)

- 1 initialise $\hat{Q}$, counts $C \leftarrow 0$, and $g(s) \leftarrow \arg \max_a \hat{Q}(s, a)$
- 2 **for** each episode: draw (s_0, a_0) at random, every feasible pair with positive probability
- 3 simulate T periods: first action a_0 , then follow g
- 4 $G \leftarrow 0$; **for** $t = T - 1, \dots, 0$: $G \leftarrow r_{t+1} + \gamma G$
- 5 **if** (s_t, a_t) is a first visit: $C \leftarrow C + 1$, $\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + (G - \hat{Q}(s_t, a_t)) / C$
- 6 for every s : $g(s) \leftarrow \arg \max_{a \in \mathcal{A}(s)} \hat{Q}(s, a)$

- **Exploring starts:** reset the economy to a random (k_0, z_0) and a random first k_1 . A structural simulator allows that. A real economy does not.
- Only the first step is random. States far from the start stay rare, and with many states random starts waste samples.
- **Clean in theory, and it needs a reset button.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §5.3; Miao & Feng, *AI for Economists*, ch. 11, “Exploring starts”.

ϵ -Greedy: Randomness as a Learning Tool

- ▶ **ϵ -greedy**: with probability $1 - \epsilon$ take the action that looks best; with probability ϵ draw uniformly from the feasible actions. With $\mathcal{A}^*(s) = \arg \max_a \hat{Q}(s, a)$,

$$g_\epsilon(a | s) = \frac{\epsilon}{|\mathcal{A}(s)|} + \frac{1 - \epsilon}{|\mathcal{A}^*(s)|} \mathbf{1}\{a \in \mathcal{A}^*(s)\}.$$

- ▶ Every feasible action keeps probability at least $\epsilon/|\mathcal{A}(s)|$. A state visited again and again has every action tried again and again. No reset is needed.
- ▶ In the growth model the action is next period's capital, so trying other actions moves the economy to other states.
- ▶ The optimal rule is deterministic, and finding it is still the goal. The randomness is there to expose errors in $\hat{Q}$; it can shrink as $\hat{Q}$ improves.
- ▶ **Two prices: you learn the value of a rule that trembles, and only where the economy actually goes.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “ ϵ -greedy”.

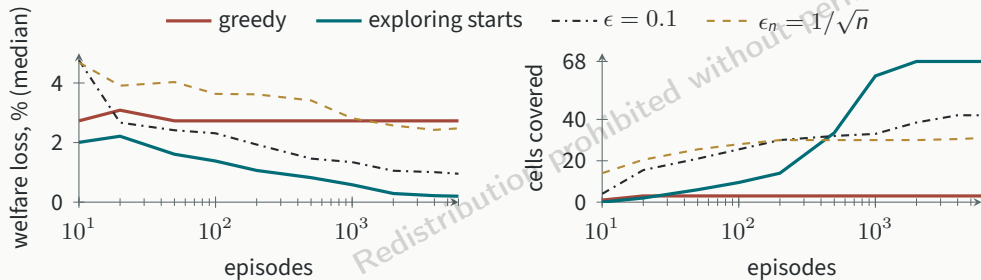
ϵ -Greedy Monte Carlo Control

First-visit Monte Carlo control, on the rule's own data

- 1 initialise $\hat{Q}$, counts $C \leftarrow 0$; choose a start s_0 (it may be fixed) and a schedule ϵ_i
 - 2 **for** episode $i = 1, \dots, N$:
 - 3 from s_0 , draw every action from g_{ϵ_i} built on the current $\hat{Q}$; simulate T periods
 - 4 $G \leftarrow 0$; **for** $t = T - 1, \dots, 0$: $G \leftarrow r_{t+1} + \gamma G$
 - 5 **if** (s_t, a_t) is a first visit: $C \leftarrow C + 1$, $\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + (G - \hat{Q}(s_t, a_t)) / C$
 - 6 **return** $\hat{Q}$ and the greedy rule $g(s) = \arg \max_a \hat{Q}(s, a)$ for deployment
- ▶ Three changes from exploring starts: the start may be fixed; every action is drawn from g_ϵ ; the improved rule is g_ϵ itself.
 - ▶ The rule that collects the data trembles. The rule that is deployed at the end does not.
 - ▶ **Line 5 is a running mean. Deck 3.4 starts from that one line.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §5.4; Miao & Feng, *AI for Economists*, ch. 11, “ ϵ -greedy”.

Measured on the Whole Model: Greedy, Exploring Starts, ϵ -Greedy



- ▶ After 6,000 episodes: greedy 2.73%, 3 cells covered (10 or more samples); exploring starts 0.20%, all 68; $\epsilon = 0.1$ from a fixed start 0.95%, 42.
- ▶ **Loss: the permanent rise in consumption that would compensate for using the learned rule.**

Source: Recorded by the book's script: medians of 20 seeds, one path per cell to start; 7×2 model, $\beta = 0.95$.

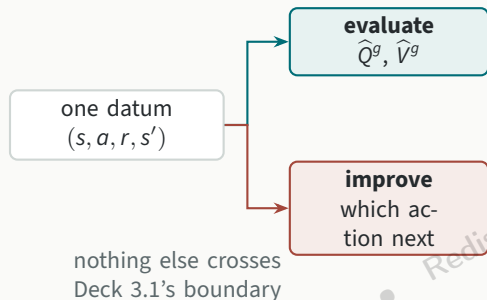
Session 3.3

Who Generates the Data

What a datum is, what it is for, and whose value you end up with



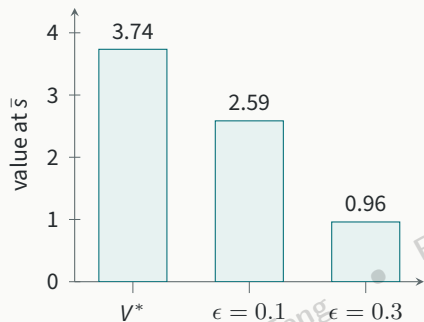
What the Data Are, and What They Are For



- ▶ A datum is one tuple, and it does exactly two jobs. Every method this session is a choice of how to do each.
- ▶ On-policy or off-policy is one question about that pair: **is the rule that generated the tuple the rule being valued?**
- ▶ The economist's version: your sample came from one policy regime and you want the value of another.
- ▶ **Ask of any result: which rule produced these data, and whose value is being reported?**

Source: Miao & Feng, *AI for Economists*, ch. 11, "On-policy and off-policy".

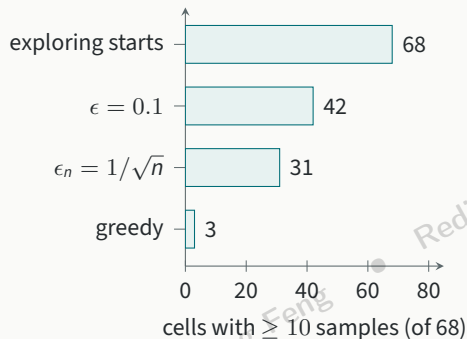
On-Policy and Off-Policy



- **Behaviour policy** b : generates the data. **Target policy** g : the one being learned. Same: **on-policy**. Different: **off-policy**.
- On-policy with $\epsilon > 0$ learns the value of the trembling rule: 5.91% of consumption lost at $\epsilon = 0.1$, 14.88% at 0.3.
- The gap between best and second best shrinks from 0.224 to 0.084.
- **In a real economy trembles are lived. Economists explore inside a simulator.**

Source: Best ϵ -soft rule at $\bar{s} = (3.75, 0.9)$, recorded by the book's script; Miao & Feng, *AI for Economists*, ch. 11.

Data Are Generated by the Policy



- ▶ The rule you follow decides what you see. Its **visitation distribution** $d_g(s, a)$ is the discounted frequency of each pair.
- ▶ Where $d_g(s, a) = 0$, $Q(s, a)$ is not identified: the overlap condition of causal inference.
- ▶ One state alone is a **multi-armed bandit**. Rothschild's firm can stay at a wrong price for ever.
- ▶ **Exploration buys identification.**

Source: Recorded by the book's script (6,000 episodes, medians). M. Rothschild, *JET* 9(2), 1974; Miao & Feng, ch. 11.

Importance Sampling: Reweighting Someone Else's Data

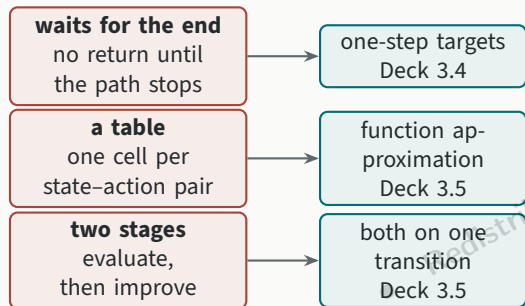
- ▶ Collect with an exploring b , value a greedy g , and correct the mismatch with the **importance ratio** $\rho_{t:T-1} = \prod_{j=t}^{T-1} g(a_j | s_j) / b(a_j | s_j)$.
- ▶ An economist knows it: inverse probability weighting, with the **coverage condition** $g > 0 \Rightarrow b > 0$ as overlap. The weighted form is Hájek's.

horizon T	target	behaviour	ordinary IS	weighted IS	paths with weight > 0
5	0.627	0.621	0.628 (0.026)	0.628 (0.017)	65.9%
20	2.286	1.748	2.312 (0.218)	2.292 (0.072)	19.0%
50	3.424	2.382	3.437 (1.303)	3.422 (0.285)	1.6%

- ▶ One tremble anywhere sets a path's weight to zero. At $\epsilon = 0.3$ and $T = 50$, no usable path is left.
- ▶ **The ratio is a product over the whole path, and the product is the problem. Deck 3.4's Q-learning is off-policy one step at a time, with no product at all.**

Source: Horvitz & Thompson, *JASA* 47, 1952; recorded by the book's script: from $\bar{s}$, 500 paths per estimate, 200 repetitions.

What Tabular Monte Carlo Cannot Do



- ▶ The book closes the chapter with three limits, each naming a piece of what comes next: it must wait for a trajectory to end; a table cannot be stored in high dimension; and evaluation and improvement run as two stages.
- ▶ The third is a way of teaching the method, not a requirement of the loop.
- ▶ **Variance is not on this list;** it belongs to the next chapter.
- ▶ **Next, Deck 3.4: update after one step, and stop waiting for the path to end.**

Source: Miao & Feng, ch. 11, §11.7, the bridge to ch. 12; variance, ch. 12.

Session 3.4

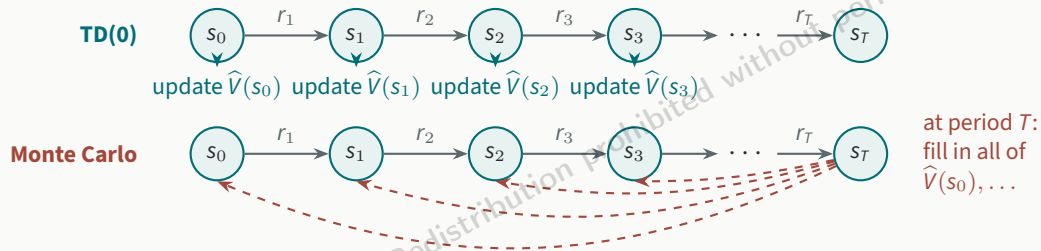
Temporal Difference: TD(0), SARSA, Q-Learning

Update after every step by leaning on the estimate you already have: when that is safe, and what it costs



Zhigang Feng

Monte Carlo Waits for the Whole Path. Temporal Difference Does Not



- ▶ Deck 3.3 averaged whole returns, $T = 300$: each sample waits T periods and carries every shock on the way.
- ▶ **This deck takes $T = 1$: one real reward, and the current estimate for everything after.**

Source: After the update-timing figure in Miao & Feng, *AI for Economists*, ch. 12, "TD(0)".

Session 3.4

Evaluation, One Step at a Time

The incremental mean, TD(0), and everything between one step and the whole path



The Sample Mean, Written One Observation at a Time

- ▶ The mean of $n + 1$ draws is the old mean plus a correction:

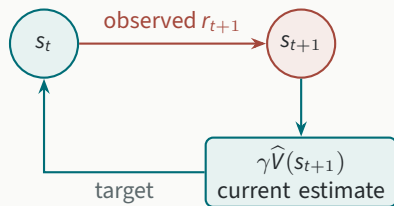
$$\bar{X}_{n+1} = \bar{X}_n + \frac{1}{n+1} (X_{n+1} - \bar{X}_n).$$

New estimate = old estimate + step size $\times$ (observation – old estimate). Deck 3.3's $\hat{Q}$ was updated exactly so.

- ▶ Replace $1/(n+1)$ by a general **step size** η_n . The **Robbins–Monro conditions** $\sum_n \eta_n = \infty, \sum_n \eta_n^2 < \infty$ give convergence: $\eta_n = n^{-\omega}$ with $\frac{1}{2} < \omega \leq 1$ qualifies.
- ▶ A constant η fails the second condition: the estimate hovers around the mean, with variance $\eta\sigma^2/(2-\eta)$.
- ▶ **A constant step forgets old data at rate $1 - \eta$. That is a virtue when the target itself moves.**

Source: H. Robbins, S. Monro, *Annals of Mathematical Statistics* 22(3), 1951; Miao & Feng, *AI for Economists*, ch. 12.

TD(0): One Real Reward, the Rest on Credit



- After each transition $(s_t, a_t, r_{t+1}, s_{t+1})$:

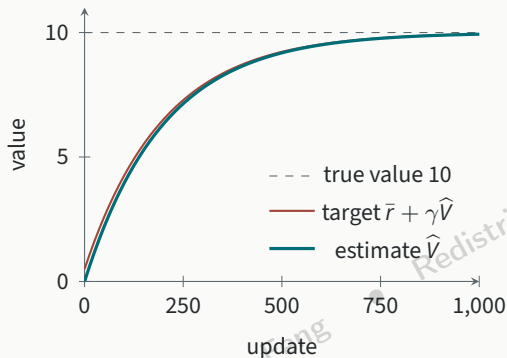
$$\hat{V}(s_t) \leftarrow \hat{V}(s_t) + \eta \delta_t,$$

$$\delta_t = r_{t+1} + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t).$$

- $r_{t+1} + \gamma \hat{V}(s_{t+1})$: the **TD target**. δ_t : the **TD error**.
- **Bootstrapping**: the estimate stands in for the unseen future.
- **Against DP: no sum over z' . Against Monte Carlo: no waiting.**

Source: R. Sutton, "Learning to predict by the methods of temporal differences", *Machine Learning* 3, 1988; Miao & Feng, ch. 12.

A Biased Target That Still Converges



- ▶ One state, reward 0.5 every period, $\gamma = 0.95$: true value 10. Start at $\hat{V} = 0$, $\eta = 0.1$.
- ▶ First three estimates: 0.050, 0.100, 0.149. The first target is only 0.5, but it is *above* the estimate.
- ▶ The expected target is $T^g \hat{V}$, a γ -contraction: always nearer the truth than $\hat{V}$ is.
- ▶ **Estimate and target rise together; the gap shrinks by $1 - \eta(1 - \gamma) = 0.995$ per update.**

Source: Arithmetic, after the worked example in Miao & Feng, *AI for Economists*, ch. 12, “Why a biased target still converges”.

TD(0) Policy Evaluation

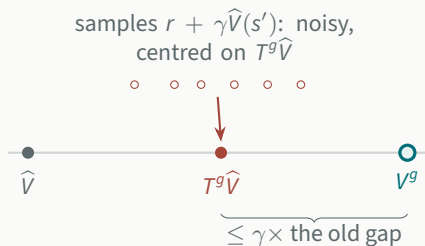
TD(0): evaluate a given policy g along one simulated path

- 1 initialise $\hat{V}(s)$; visit counts $C(s) \leftarrow 0$; $s \leftarrow s_0$
- 2 **for** $t = 0, 1, 2, \dots$ up to the step budget:
- 3 draw $a \sim g(\cdot | s)$; the simulator returns r and s'
- 4 $C(s) \leftarrow C(s) + 1$
- 5 $\delta \leftarrow r + \gamma \hat{V}(s') - \hat{V}(s)$ keep the bootstrap term even where a path is cut
- 6 $\hat{V}(s) \leftarrow \hat{V}(s) + \eta \delta$ η constant, or $C(s)^{-\omega}$
- 7 $s \leftarrow s'$

- ▶ Theorem: in a finite MDP, if every state is visited infinitely often and the step sizes satisfy Robbins–Monro, $\hat{V} \rightarrow V^g$ with probability one.
- ▶ The theorem does not say which step size is fast. $1/C(s)$ qualifies, and is unusably slow.
- ▶ **No episodes are needed: the path just runs. Cutting it is not ending it.**

Source: J. Tsitsiklis, *Machine Learning* 16, 1994; Jaakkola, Jordan & Singh, *Neural Computation* 6(6), 1994; Miao & Feng, ch. 12.

TD(0) with Indicator Features: a Table Is the Special Case



- ▶ A table is a linear critic: let $\phi(s)$ be the indicators $\mathbf{1}\{s = s_j\}$, so that ψ_j is the cell for s_j .
- ▶ Three things make it work: T^g contracts; samples centre on $T^g \hat{V}$; $C(s)^{-0.6}$ meets Robbins–Monro and restarts keep every state visited.
- ▶ Indicators share nothing, so the best fit is the target. Coarser features (Deck 3.5) give that up.
- ▶ **Measured below, fixed rule, 7×2 grid:**
0.044 at 10^5 steps, 0.028 from a 4.27 start.

Source: D. Bertsekas, J. Tsitsiklis, *Neuro-Dynamic Programming*, 1996, ch. 4; errors recorded by the book's script for ch. 12, 7×2 grid.

n -Step Returns: Between One Step and the Whole Path



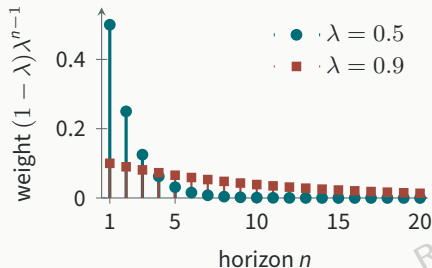
- **Deck 3.2 asked how far to roll out.** Its answer was flat: there, only $n = 1$ learned.
- The **n -step return**: n real rewards, then the estimate,

$$\hat{G}_{t:t+n} = \sum_{j < n} \gamma^j r_{t+j+1} + \gamma^n \hat{V}(s_{t+n}).$$

- $n = 1$ is TD(0); large n is Monte Carlo. Larger n : the weight γ^n falls — less bias, more variance.
- **And the update must wait n periods.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, ch. 7; Miao & Feng, *AI for Economists*, ch. 12, “ n -step returns and TD(λ)”.

The λ -Return, and the Trace That Computes It Backwards



- Deck 3.2 had to pick one n ; average every horizon at once instead:

$$\hat{G}_t^\lambda = (1 - \lambda) \sum_{n \geq 1} \lambda^{n-1} \hat{G}_{t:t+n}.$$

$\lambda = 0$: TD(0). $\lambda \rightarrow 1$: Monte Carlo.

- That needs the whole future, so take the same weights from the past:

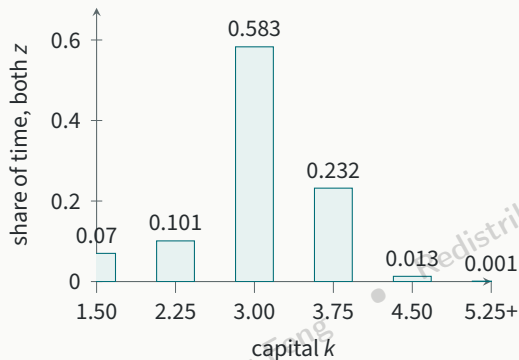
$$e_t(s) = \gamma \lambda e_{t-1}(s) + \mathbf{1}\{s_t = s\},$$

moving every state by $\eta \delta_t e_t(s)$.

- **TD(λ): one dial, and today's surprise charged to the states that led here.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, ch. 12; Singh & Sutton, *Machine Learning* 22, 1996; Miao & Feng, ch. 12.

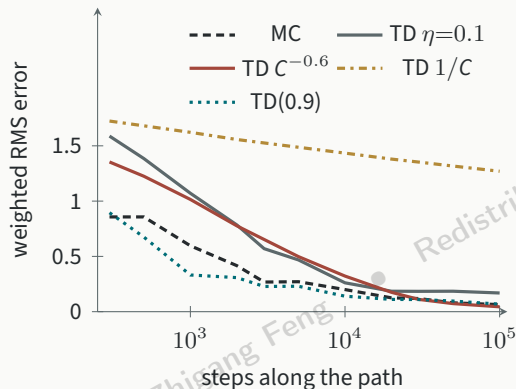
The Experiment: One Path, Five Estimators



- ▶ Evaluate g_ϵ : the optimal rule, with a uniformly random feasible action 10% of the time. Its exact values run from -0.32 to 4.56 .
- ▶ One path of 100,000 steps from $(3.75, 0.9)$; every method sees only that path. Twenty paths; medians.
- ▶ Error: root mean square, weighted by where g_ϵ spends its time.
- ▶ **Under g_ϵ , 58% of the time is spent at $k = 3.00$. The economy visits few states.**

Source: Book ch. 12 script, 7×2 grid; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2.

TD against Monte Carlo, Measured



- ▶ At 1,000 steps Monte Carlo leads (0.596 against 1.071): its target contains no estimate.
- ▶ At 100,000 steps TD(0) with $\eta = C(s)^{-0.6}$ leads: 0.044 against 0.064.
- ▶ TD(0.9) cuts the early error to a third of TD(0)'s, 0.332. The edge is gone by 100,000.
- ▶ **Bias dominates early, variance late.**

Source: Recorded by the book's script for ch. 12; initial value 0, medians of 20 paths; 7×2 grid, $\beta = 0.95$.

Target and Step Size Cannot Be Judged Apart

steps	initial value 0			initial value 4.27		
	1,000	10,000	100,000	1,000	10,000	100,000
Monte Carlo, $\eta = 1/C(s)$	0.596	0.200	0.064	0.586	0.183	0.060
TD(0), $\eta = 0.1$	1.071	0.261	0.169	1.394	0.142	0.158
TD(0), $\eta = C(s)^{-0.6}$	1.015	0.322	0.044	1.025	0.245	0.028
TD(0), $\eta = 1/C(s)$	1.622	1.434	1.270	1.690	1.484	1.319
TD(0.9), $\eta = C(s)^{-0.6}$	0.332	0.140	0.068	0.293	0.132	0.066

- ▶ $1/C(s)$ suits Monte Carlo's unbiased targets; for TD it weights early, poor targets like late, good ones. The start 4.27 is steady-state consumption valued for ever: a guess needing no data.
- ▶ Deck 3.2's $n = 5$ collapse is this one level up: its *rule* moved as the return played out, so the long target carried the rule's error, not the estimate's.
- ▶ **A sensible start and a slowly falling step size matter as much as the target.**

Source: Recorded by the book's script for ch. 12, 7×2 grid. E. Even-Dar, Y. Mansour, *JMLR* 5, 2003, on step sizes.

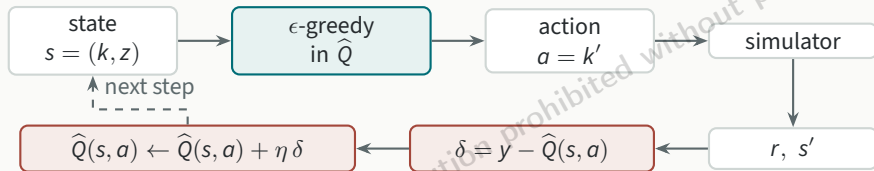
Session 3.4

Control: SARSA and Q-Learning

Improve the rule at every step, and ask whose value is being learned



From Evaluation to Control: Improve at Every Step



- Evaluation updates every step, so improvement need not wait: act on $\hat{Q}$, update it, act again.
- The object is Q again: comparing actions within a row needs no transition probabilities.
- **One choice is left: the value placed on tomorrow inside the target y .**

Source: After the control-loop figure in Miao & Feng, *AI for Economists*, ch. 12, “Temporal-difference control”.

SARSA, Derived: What an On-Policy Target Converges To

- ▶ Deck 3.2 named this road and had no table to take it: there the state and the action are both real numbers. The 7×2 grid has cells. The target values the action that will really be taken,

$$y_{t+1} = r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}), \quad a_{t+1} \sim g_{\epsilon}(\cdot \mid s_{t+1}).$$

- ▶ Average over that draw: the expected target is $T^{g_{\epsilon}} \hat{Q}$, whose fixed point is $Q^{g_{\epsilon}}$. **SARSA** converges to the value of the rule that generated the data — **on-policy**.
- ▶ With ϵ held at 0.1 that rule is not g^* but the best **ϵ -soft policy**: on the 7×2 grid that limit alone costs 0.18%, before any learning error.
- ▶ So the defect is the estimand, not the data. On the lab's 20×2 grid the gap survives: 0.69–1.47% against Q-learning's 0.36–0.39%, three seeds.
- ▶ **To reach g^* , ϵ must fall to zero slowly: greedy in the limit, with infinite exploration (GLIE).**

Source: Rummery & Niranjan, 1994; Singh et al., *Machine Learning* 38, 2000; 20×2 numbers from the book's script, ch. 12.

Q-Learning, Derived: What the Maximum Buys, and What It Costs

- ▶ One character separates it from SARSA: the next action is maximised, over the feasible set alone,

$$y_{t+1} = r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} \hat{Q}(s_{t+1}, a').$$

- ▶ Its expected target is $T^* \hat{Q}$, a γ -contraction with fixed point Q^* : greedy **target policy**, ϵ -greedy **behaviour policy**, hence **off-policy**. A fixed $\epsilon > 0$ costs nothing, given bounded rewards, Robbins–Monro steps, and every feasible pair updated for ever.
- ▶ So the defect is the data, not the estimand: where the path rarely goes a cell stays wrong — 4 states of 14 at 10^6 steps.
- ▶ **Maximisation bias**: $E[\max_{a'} \hat{Q}] \geq \max_{a'} E[\hat{Q}]$, so noise lifts the target and bootstrapping passes it on. Large when the actions are many.
- ▶ **Off-policy buys the right target, not the right data: the best portfolio's value, learned while holding a cautious one.**

Source: C. Watkins, P. Dayan, “Q-learning”, *Machine Learning* 8, 1992; Miao & Feng, *AI for Economists*, ch. 12.

Why Q-Learning Needs No Importance Ratio

- ▶ Deck 3.3: data from one rule, used to value another, must be reweighted by g/b at every period of the return.
- ▶ One-step Q-learning uses a single transition. Given (s_t, a_t) , the next state comes from $P(\cdot \mid s_t, a_t)$: the behaviour rule plays no part.
- ▶ The $\max$ draws no action from the behaviour rule at all.
- ▶ The only action it drew is a_t , and the update is *for* the cell (s_t, a_t) . Conditional on it, no ratio appears.
- ▶ **The behaviour rule only decides which cells get data. That still matters, as the measurements will show.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “Q-learning”; Watkins & Dayan, *Machine Learning* 8, 1992.

One Algorithm, One Line of Difference

TD control: SARSA and Q-learning

- 1 initialise $\hat{Q}(s, a)$ on feasible cells; counts $C(s, a) \leftarrow 0$; $s \leftarrow s_0$
- 2 choose a in $\mathcal{A}(s)$, ϵ -greedy in $\hat{Q}$
- 3 **for** $t = 0, 1, 2, \dots$ up to the step budget:
- 4 take a ; the simulator returns r and s'
- 5 choose a' in $\mathcal{A}(s')$, ϵ -greedy in $\hat{Q}$
- 6 **SARSA:** $y \leftarrow r + \gamma \hat{Q}(s', a')$ **Q-learning:** $y \leftarrow r + \gamma \max_{a'' \in \mathcal{A}(s')} \hat{Q}(s', a'')$
- 7 $C(s, a) \leftarrow C(s, a) + 1$; $\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + C(s, a)^{-0.6} [y - \hat{Q}(s, a)]$; $(s, a) \leftarrow (s', a')$

Line 6 is the whole difference. Deploy $g(s) = \arg \max_{a \in \mathcal{A}(s)} \hat{Q}(s, a)$ in both: **exploration is for learning, not for use.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “Q-learning” (algorithm for TD control). This is the lab’s block C.

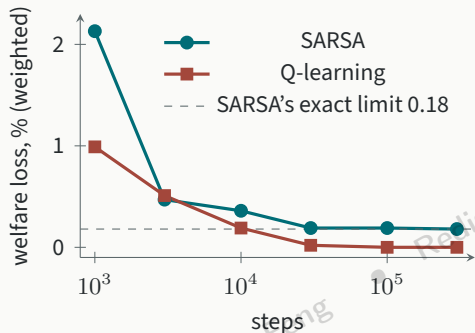
What Each Converges To: Q^* against Q_ϵ

next capital k' , from state $(k, z) = (2.25, 1.1)$	1.50	2.25	3.00
Q^* : what Q-learning approaches	1.799	2.524	2.546
Q_ϵ : what SARSA approaches, $\epsilon = 0.1$	1.000	1.676	1.605

- ▶ Both limits can be computed exactly. Their greedy choices differ in 2 of 14 states, and the same way: SARSA's limit saves one grid step less.
- ▶ Raising k' from 2.25 to 3.00 costs 0.919 of utility today. The discounted gain tomorrow is 0.941 under g^* , worth it; only 0.848 under the ϵ -soft rule, not worth it.
- ▶ Every future period carries a 10% chance of a random k' , so part of today's extra capital will be spent at random. Exploration noise dilutes the return to saving, at every grid step.
- ▶ **At $\epsilon = 0.3$ the rules differ in 4 states. All four save less.**

Source: Book ch. 12 script, 7×2 grid; $\alpha = 0.36, \beta = 0.95, \delta = 0.10, u = \ln c, z \in \{0.9, 1.1\}$ with switching probability 0.2.

Measured: 300,000 Steps, Twenty Runs



at 300,000 steps, medians	SARSA	Q
loss, weighted	0.18%	0.00%
loss, plain mean, 14 states	1.66%	2.01%
states with a wrong action	4	4
cells with 10 or more samples	43	42

- By 10,000 steps: 0.36% and 0.19%, weighted; Q-learning's last error goes by 100,000. Its 0.00% is the row max Deck 3.2 had no table for.
- **The 2% plain mean comes from $k \geq 4.50$: under 1.5% of the time.**

Source: Recorded by the book's script for ch. 12, 7×2 grid, $\epsilon = 0.1$, 20 runs. Loss: consumption-equivalent, against g^* .

Session 3.4

Using Samples Well

Exploration pace, lower-variance targets, and where TD sits among all backups



Sampling and Step Sizes: Five Fixes, One Idea

- ▶ **Pace.** A fixed ϵ parks SARSA at the ϵ -soft optimum; $\epsilon = 1/\sqrt{C(s)}$ satisfies GLIE. Boltzmann (Deck 3.3) wastes fewer trials on plainly bad actions.
- ▶ **Start. Optimistic initial values:** 4.27 exceeds most true values, so untried actions look good and get tried.
- ▶ **Step.** $\eta = 1/C$ can need steps growing exponentially in $1/(1 - \gamma)$; $C^{-\omega}$, $\frac{1}{2} < \omega < 1$, polynomially. The lab uses $\omega = 0.6$.
- ▶ **Target. Expected SARSA** replaces the drawn a_{t+1} by $\sum_{a'} g_{\epsilon}(a' | s_{t+1}) \hat{Q}(s_{t+1}, a')$: same expectation, less variance. The greedy rule in that sum is Q-learning.
- ▶ **Old data.** Multi-step off-policy targets need importance ratios, or a cut at the first non-greedy action; from history, the behaviour rule must itself be estimated.
- ▶ **Two estimators.** One table picks the action, an independent one values it: the max bias goes. **Every fix is where samples come from, or how a target uses them.**

Source: Sutton & Barto, 2018, §6.6; Even-Dar & Mansour, *JMLR* 5, 2003; H. van Hasselt, *NeurIPS*, 2010; Imbens & Rubin, 2015.

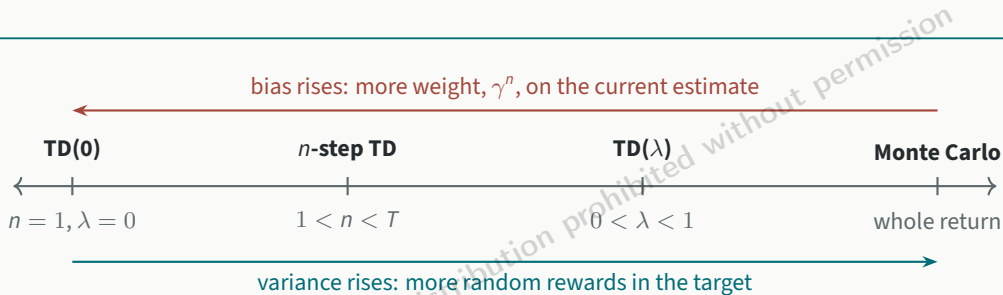
The Backup Spectrum: Two Choices Define an Evaluation Method

real rewards	full expectation over s' (needs P)	one sampled s' (no P)
one period	one DP backup: a sweep of value iteration	TD(0) : target $r + \gamma \hat{V}(s')$
n periods	modified policy iteration: roll the rule forward n periods	n-step TD, TD(λ)
the whole path	exact evaluation: solve the linear system	Monte Carlo : the return G_t alone

- ▶ Columns: is P known? Rows: how much the target leans on the current estimate.
- ▶ **Decks 3.1 to 3.4 filled this table. Every cell is the same Bellman equation.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “One mathematical skeleton, different solution paths” (table).

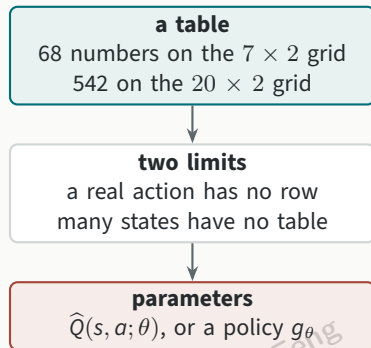
One Dial: Bias against Variance



- ▶ TD(0): one random reward, low variance; a poor estimate leaks into every target.
- ▶ Monte Carlo: no estimate in the target, so no such bias; every shock along the path is in it.
- ▶ **Deck 3.2's actor-critic sat at the TD end from its first line: an update every step, driving the rule as it goes.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "One mathematical skeleton"; Sutton & Barto, 2018, ch. 7 and 12.

What a Q Table Cannot Do



- ▶ Done so far: the expectation is sampled (3.3), the update is one step (3.4), evaluation and improvement interleave.
- ▶ **Two limits, not one.** A real action has no row to max over; many states have no table to hold — challenge 1 of Deck 3.1, untouched here.
- ▶ Deck 3.2 stepped past both: a gradient instead of a max, five weights instead of cells. But every theorem here assumes the table.
- ▶ **Deck 3.5 says when that is safe — linear features, then a network — and why a nudge of $\delta \nabla \log g_\theta$ points uphill.**

Source: Cell counts: Session 3 lab, block A. Miao & Feng, *AI for Economists*, ch. 12, “Function approximation”.

Session 3.5

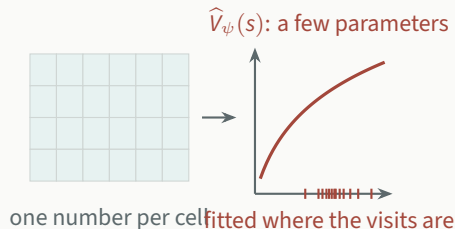
From Tables to Networks: Function Approximation, Policy Gradient, Actor-Critic

Parameters replace the table; then a gradient step replaces the max



Zhigang Feng

Where a Table Stops Fitting, and What Deck 3.2's Critic Already Did



- ▶ Deck 3.4 kept a table of 68, one number per feasible cell. Deck 3.2's critic never had cells: $\hat{V}_\psi = \psi^\top \phi$, **five numbers**.
- ▶ Capital is continuous; with heterogeneous agents the state holds a distribution.
- ▶ **Function approximation**: far fewer parameters than states — fitted where the samples fall, not at nodes you placed. A table is its indicator-feature case.
- ▶ **Two routes: a basis an economist picks, then features a network learns. Then the max goes too.**

Source: Miao & Feng, *AI for Economists*, ch. 12; K. Judd, *Numerical Methods in Economics*, 1998, ch. 11.

Session 3.5

Route One: Linear in Parameters

A basis the economist picks — where the theory still holds, and what it promises



Linear Value Functions and Semi-Gradient TD

- Pick p **basis functions**: $\hat{V}_\psi(s) = \psi^\top \phi(s)$, linear in ψ , not in the state. With $\tilde{k}$ capital rescaled to $[0, 1]$:

$$\phi(k, z) = (1, \tilde{k}, \tilde{k}^2, \mathbf{1}\{z = 1.1\}, \tilde{k} \cdot \mathbf{1}\{z = 1.1\})^\top.$$

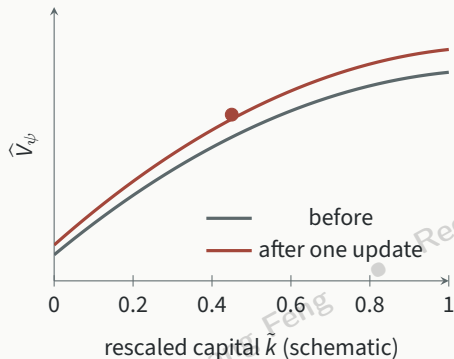
- The TD error $\delta_t = r_{t+1} + \gamma \hat{V}_\psi(s_{t+1}) - \hat{V}_\psi(s_t)$ is unchanged. There is no cell to correct, so move ψ :

$$\psi \leftarrow \psi + \eta \delta_t \nabla_\psi \hat{V}_\psi(s_t) = \psi + \eta \delta_t \phi(s_t).$$

- **Semi-gradient**: the target depends on ψ too, but is held constant (detached, in PyTorch).
- **A table is this with indicator features, one per cell: then $\phi(s_t)$ selects one entry and nothing else moves.**

Source: R. Sutton, A. Barto, *Reinforcement Learning*, 2018, §9.3; Miao & Feng, *AI for Economists*, ch. 12.

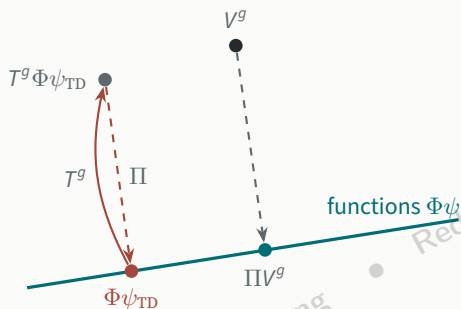
One Update Moves Many States



- ▶ A table corrects one cell. A step on ψ shifts every state that shares $\tilde{k}$ and $\tilde{k}^2$.
- ▶ Gain: one visit updates a neighbourhood; unvisited states get an estimate.
- ▶ Risk: a poor basis pushes error onto states that did nothing wrong.
- ▶ **Generalisation is the point, and the source of the failures.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “Linear approximation and semi-gradient TD”. Curves are schematic.

Where Linear TD Converges: the Projected Bellman Equation



- ▶ On-policy data, full-rank features: ψ converges to the **TD fixed point** $\Phi\psi = \Pi T^g \Phi\psi$.
- ▶ Π : least squares weighted by the stationary distribution $\bar{d}_g$.
- ▶ Not the best fit, but within $\frac{1}{1-\gamma}$ of it in the $\bar{d}$ -norm.
- ▶ **At $\gamma = 0.95$ that factor is 20; a sharper argument gives 3.2. Off-policy data void the theorem.**

Source: J. Tsitsiklis, B. Van Roy, *IEEE Trans. Automatic Control* 42(5), 1997, Theorem 1; $(1 - 0.95^2)^{-1/2} = 3.2$.

The TD Fixed Point Is an Instrumental-Variables Estimator

- ▶ The fixed point is a moment condition under the stationary distribution:

$$E \left[\phi(s_t) (r_{t+1} + \gamma \phi(s_{t+1})^\top \psi - \phi(s_t)^\top \psi) \right] = 0.$$

- ▶ As a regression, $r_{t+1} = (\phi(s_t) - \gamma \phi(s_{t+1}))^\top \psi + e_{t+1}$: the regressor contains s_{t+1} , whose randomness is also in e_{t+1} . OLS is biased; $\phi(s_t)$, fixed beforehand, is a valid instrument.
- ▶ Solving the sample moments gives **least-squares TD** (LSTD):

$$\hat{\psi}_{\text{LSTD}} = \left[\sum_t \phi(s_t) (\phi(s_t) - \gamma \phi(s_{t+1}))^\top \right]^{-1} \sum_t \phi(s_t) r_{t+1}.$$

- ▶ **Semi-gradient TD walks towards this solution step by step. LSTD solves for it at once.**

Source: S. Bradtke, A. Barto, *Machine Learning* 22, 1996; J. Boyan, *Machine Learning* 49, 2002; Miao & Feng, ch. 12.

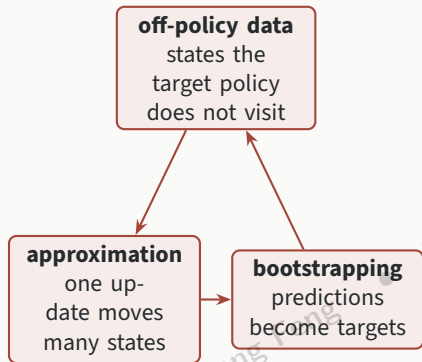
Measured: Three Linear Bases, One Policy

basis	weighted error		max error	bound	
	best fit	TD point	TD point	theorem	sharper
5 terms, polynomial in $\tilde{k}$	0.025	0.027	0.902	0.493	0.079
4 terms, in $\ln k$	0.023	0.023	0.066	0.459	0.074
3 terms, no curvature	0.144	0.165	1.738	2.887	0.462

- ▶ Recorded on the 7×2 grid, evaluating Deck 3.4's ϵ -greedy rule: errors in value units, not consumption.
- ▶ Rows 1 and 2: same weighted error, maximum error 14 times apart. With log utility, economics says use $\ln k$.
- ▶ **On 200,000 steps LSTD errs by 0.030; semi-gradient TD, at the same point, is at 0.190.**
- ▶ *Continuous model*: Deck 3.2's five-feature critic ends at rms 1.72 against V^* — a poor fit, on a rule that still scores 0.014%. **A critic is a means, not an end.**

Source: The book's script for ch. 12, 7×2 grid, errors weighted by $\bar{d}$; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2. Continuous model: Session 3 lab, block A2.

The Deadly Triad



- ▶ Function approximation, bootstrapping and off-policy sampling: any two are safe, all three can diverge. Baird's star is the standard counter-example.
- ▶ Linear TD on the rule's own data has only the first two — which is why Deck 3.2's critic has a guarantee at all. **Deck 3.2 is not in the triad**: a linear critic, on its own path.
- ▶ **With a neural critic there is no general global convergence result**, and the book says so of its own chapter's algorithms.

Source: L. Baird, "Residual algorithms", *ICML*, 1995; Sutton & Barto, 2018, §11.3; Miao & Feng, ch. 12, §12.6 footnote.

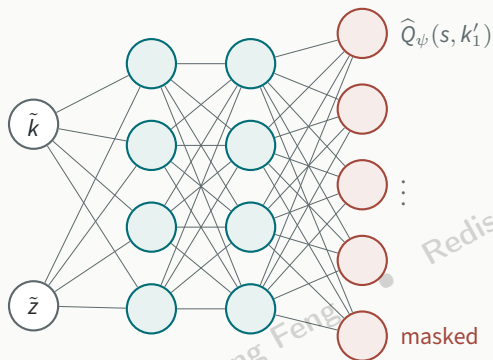
Session 3.5

Route Two: Learn the Features

A network for Q , then the policy itself — and then stop taking the max



Let the Features Be Learned: a Network for Q

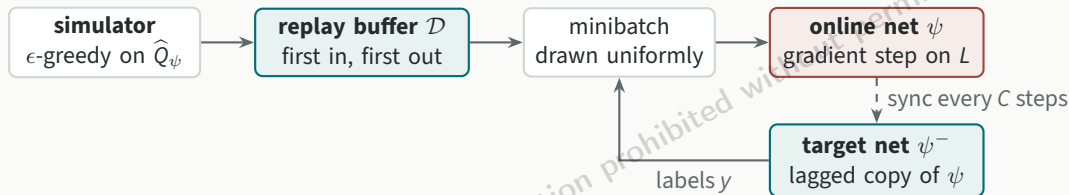


hidden layers: the learned features

- ▶ A linear basis is as good as the economist's guess. A network's early layers *are* the basis, and they are trained.
- ▶ One output per action k' ; infeasible ones are masked before the max.
- ▶ In Q-learning this completes the triad: it oscillates or diverges.
- ▶ **DQN adds two devices that hold the regression still.**

Source: V. Mnih et al., "Human-level control through deep reinforcement learning", *Nature* 518, 2015.

DQN: a Target Network and a Replay Buffer

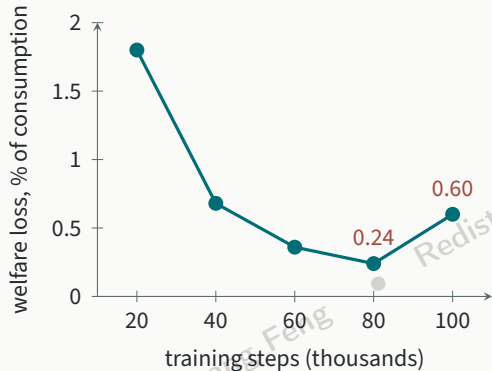


$$y_t = r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} \hat{Q}_{\psi^-}(s_{t+1}, a'), \quad L(\psi) = E_{\mathcal{D}} [(y - \hat{Q}_\psi(s, a))^2].$$

- **Target network:** a frozen label, so the regression has a fixed target. It acts on bootstrapping.
- **Experience replay:** decorrelates successive samples and reuses them. It does not repair the off-policy mismatch.
- **Neither removes the triad. They make training stable enough, with tuning.**

Source: Mnih et al., *Nature* 518, 2015; Miao & Feng, *AI for Economists*, ch. 12, “Neural networks and DQN”.

DQN on the Growth Model, Recorded



- ▶ A network for $\hat{Q}$ on the 20×2 grid, 40 states: two hidden layers of 64 units, sync every 500 gradient steps, ϵ from 1 to 0.1.
- ▶ At 100,000 steps the greedy rule matches dynamic programming in 26 of the 40 states; welfare loss 0.60%.
- ▶ Random actions lose over 100%. The book's three seeds: 0.60, 0.38, 0.20%.
- ▶ **The best iterate came at 80,000 steps, not at the end.**

Source: Recorded by the book's script for ch. 12; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2; 20×2 grid, seed 0; consumption-equivalent loss, visit-weighted.

Parameterise the Policy: a Share, or a Masked Softmax

- ▶ Double DQN and Rainbow refine that max; none removes it. A portfolio share cannot be enumerated, and $\arg \max_a \hat{Q}_\psi$ flips on a small change in ψ .
- ▶ So make the policy differentiable in θ . For discrete actions, a **masked softmax**:

$$g_\theta(a | s) = \frac{\exp h_\theta(s, a)}{\sum_{a' \in \mathcal{A}(s)} \exp h_\theta(s, a')}, \quad a \in \mathcal{A}(s),$$

zero elsewhere — **the 7×2 grid's parameterisation.**

- ▶ **For a real-valued action, put the density on the action:** Deck 3.2 draws $x \sim N(\mu_\theta, \sigma^2)$ and sets $a = \text{sigmoid}(x)$. **The theorem below covers both.**
- ▶ **Q-learning fits $\hat{Q}$ to Bellman's equation, then maximises. This maximises V^{g_θ} directly.**

Source: H. van Hasselt, A. Guez, D. Silver, AAAI, 2016; M. Hessel et al., AAAI, 2018; Miao & Feng, ch. 12, “Improving the policy directly”.

The Policy-Gradient Theorem

- ▶ If g_θ is differentiable in θ , rewards are bounded and $\gamma < 1$, then with G_t the return from t :

$$\nabla_\theta J(\theta) = E \left[\sum_{t \geq 0} \gamma^t \nabla_\theta \ln g_\theta(a_t | s_t) G_t \right] = \frac{1}{1 - \gamma} \sum_{s,a} d_{g_\theta}(s, a) \nabla_\theta \ln g_\theta(a | s) Q^{g_\theta}(s, a).$$

- ▶ $\nabla_\theta \ln g_\theta$ is the **score** of maximum likelihood: it makes a_t more likely in s_t . The return is its weight.
- ▶ First form: estimate the gradient from simulated paths. Second form: Q^{g_θ} is still inside, weighted by the **discounted visit distribution** d_{g_θ} that the γ^t builds.
- ▶ **This is the debt Deck 3.2 recorded** — row four of *Nine Numbers, Six Ideas*: improve by a gradient, not a max. **Algorithms differ only in how they estimate Q^{g_θ} : whole returns now, a TD critic later.**

Source: Sutton, McAllester, Singh & Mansour, *NeurIPS* 12, 2000; Marbach & Tsitsiklis, *IEEE TAC* 46(2), 2001.

Why the Environment Drops Out

$$P_{\theta}(\tau) = P_0(s_0) \underbrace{g_{\theta}(a_0 | s_0)}_{\text{contains } \theta} \underbrace{P(s_1 | s_0, a_0)}_{\text{does not}} \underbrace{g_{\theta}(a_1 | s_1)}_{\text{contains } \theta} P(s_2 | s_1, a_1) \dots$$

- ▶ Deck 3.2 drew this factorisation to explain why no P_z appeared in its seven lines. Here is what it proves.
- ▶ Differentiate $J = \int P_{\theta}(\tau) R(\tau) d\tau$ with $\nabla P_{\theta} = P_{\theta} \nabla \ln P_{\theta}$. In $\ln P_{\theta}(\tau)$ the simulator's factors have zero derivative: $\nabla_{\theta} \ln P_{\theta}(\tau) = \sum_t \nabla_{\theta} \ln g_{\theta}(a_t | s_t)$.
- ▶ No transition probability is needed: simulated paths are enough. Rewards earned before a_t drop out in expectation.
- ▶ **Keep the γ^t : dropping it biases the update — 10.2% in the book's network run, though the recorded tabular run's line 6 drops it and survives.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "The policy-gradient theorem"; P. Thomas, *ICML*, 2014.

REINFORCE with a Baseline

REINFORCE with a learned baseline $\hat{V}_\psi$

- 1 initialise θ, ψ ; **repeat** until the iteration limit:
- 2 draw $s_0 \sim P_0$; simulate T periods under g_θ : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$
- 3 backwards: $G_{t:T} = r_{t+1} + \gamma G_{t+1:T}$, with $G_{T:T} = 0$
- 4 $\Delta\theta \leftarrow \sum_{t < T_1} \gamma^t \nabla_\theta \ln g_\theta(a_t | s_t) (G_{t:T} - \hat{V}_\psi(s_t))$
- 5 $\Delta\psi \leftarrow \sum_{t < T_1} (G_{t:T} - \hat{V}_\psi(s_t)) \nabla_\psi \hat{V}_\psi(s_t)$
- 6 $\theta \leftarrow \theta + \eta_\theta \Delta\theta, \quad \psi \leftarrow \psi + \eta_\psi \Delta\psi$
 - A **baseline** $v(s)$ is free: $E_a[\nabla_\theta \ln g_\theta v(s)] = v(s) \nabla_\theta \sum_a g_\theta(a | s) = 0$. With $v = V^g$ the weight is the **advantage** $A^g = Q^g - V^g$.
 - Lines 4–5 sum only over $t < T_1$: with $T = 300$ -period paths, only the first $T_1 = 150$ returns are used, so every one of them has at least 150 real rewards behind it.
 - **Line 4 is Monte Carlo evaluation again: whole returns, used once, after the path ends.**

Source: R. Williams, *Machine Learning* 8, 1992; Miao & Feng, *AI for Economists*, ch. 12, “REINFORCE and baselines”.

Measured: a Baseline Is a Control Variate

baseline	summed variance		mean of 400 paths	
	value	relative	norm	distance to exact
none	141.8	1	1.378	0.618
the true value V^{g_θ}	17.3	0.122	1.489	0.228
a learned $\hat{V}$	17.4	0.123	1.492	0.229

- ▶ A uniform random policy on the 7×2 grid; exact gradient norm 1.420; one 300-period path per estimate.
- ▶ The mean is unchanged within sampling error; the variance falls to an eighth.
- ▶ **A rough baseline does as well as the true one. A control variate needs correlation, not accuracy.**

Source: Recorded by the book's script for ch. 12; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2; 7×2 grid. Greensmith, Bartlett & Baxter, *JMLR* 5, 2004.

Session 3.5

Actor-Critic

Put evaluation back inside improvement — and settle Deck 3.2's account



What REINFORCE Still Pays, and the One Change That Fixes It

REINFORCE: weight $G_{t:T} - v(s_t)$

waits for the path to end

cuts the infinite horizon at T

carries the noise of a whole path

Actor-critic: weight δ_t

δ_t is known one period later

$\gamma \hat{V}_\psi(s_{t+1})$ carries the whole tail

one reward, one transition

- ▶ With $\delta_t = r_{t+1} + \gamma \hat{V}_\psi(s_{t+1}) - \hat{V}_\psi(s_t)$ and an exact critic, $E[\delta_t \mid s_t, a_t] = A^g(s_t, a_t)$.
- ▶ This is Deck 3.2's sample backup (*Sampling the Tree*) with $T = 1$. An n -step return minus the baseline sits in between.
- ▶ **The price is bias: a wrong critic points the actor the wrong way.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “What was gained, what was lost” and “Actor-critic”.

One-Step Actor-Critic

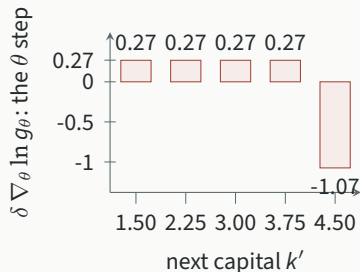
One-step actor-critic, online

- 1 initialise θ, ψ ; draw s from the start distribution
- 2 **for** $t = 0, 1, 2, \dots$ up to the step limit:
- 3 draw $a \sim g_\theta(\cdot | s)$ on the feasible set; the simulator returns r and s'
- 4 $\delta \leftarrow r + \gamma \widehat{V}_\psi(s') - \widehat{V}_\psi(s)$ (bootstrap through resets and truncation)
- 5 $\psi \leftarrow \psi + \eta_\psi \delta \nabla_\psi \widehat{V}_\psi(s)$ (critic: semi-gradient TD)
- 6 $\theta \leftarrow \theta + \eta_\theta \delta \nabla_\theta \ln g_\theta(a | s)$ (actor: policy gradient)
- 7 at a reset, redraw s ; otherwise $s \leftarrow s'$

- ▶ The **actor** is the policy; the **critic** is a value estimate trained by TD on the actor's data.
- ▶ $\delta > 0$: better than the critic expected, so a becomes more likely.
- ▶ **Evaluation and improvement take one step each, on the same transition. Deck 3.2 ran these seven lines with eight numbers and no table.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "Actor-critic"; Sutton & Barto, *Reinforcement Learning*, 2018, §13.5.

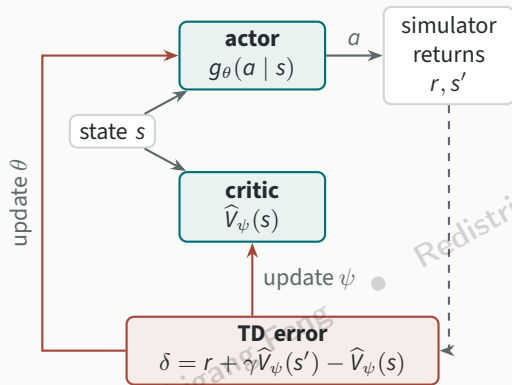
The Demo's Actor-Critic, Now Derived



- ▶ On a grid the actor is this softmax with **indicator features**: one ϕ per cell, so the score closes to $\mathbf{1}\{\cdot = a\} - g_{\theta}(\cdot | s)$ in row s , zero elsewhere.
- ▶ **The plot is the actual θ step**, score times δ : at $\delta = -1.3425$ the drawn action 4.50 falls by 1.074 and every other feasible action rises by 0.269.
- ▶ **This is the recorded 7×2 run**: the same algorithm with a 14-cell critic and a 68-weight softmax, scoring 0.52 %. Deck 3.6's ladder uses it.
- ▶ Deck 3.2's actor is the continuous cousin, with score $\frac{x-\mu}{\sigma^2} \nabla_{\theta} \mu$. **One theorem covers both.**

Source: Sutton et al., *NeurIPS* 12, 2000; A. Agarwal, S. Kakade, J. Lee, G. Mahajan, *JMLR* 22, 2021.

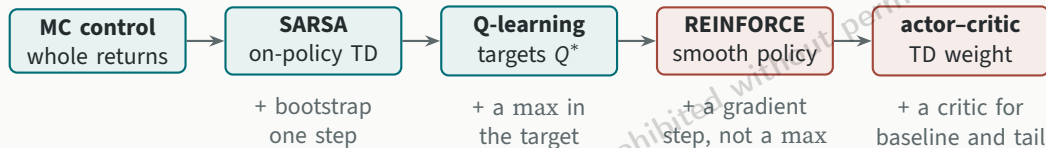
The Data Flow: One TD Error, Two Updates



- ▶ One loss in code:
 $L = \frac{1}{2}\delta^2 - \delta \ln g_{\theta}(a | s)$. Detach the target for the critic, and δ for the actor.
- ▶ The critic must keep up with a moving actor: its step size is larger.
- ▶ With a known, differentiable model the actor can take a **pathwise gradient** through the Bellman right-hand side.
- ▶ **That variant is Deck 3.6's deep value iteration.**

Source: V. Konda, J. Tsitsiklis, "Actor-critic algorithms", *NeurIPS* 12, 2000; Miao & Feng, *AI for Economists*, ch. 12.

One Ladder: from Monte Carlo Control to Actor-Critic



- ▶ Each rung changes one thing; none reads the model. Deck 3.2 climbed it backwards, and said so.
- ▶ *Tables* on the 7×2 grid, medians of 20 seeds: REINFORCE loses 1.68 %, actor-critic 0.52 %. *Networks* on the 20×2 grid, seed 0: 0.73 % and 0.19 %, with the other two seeds reversing it.
- ▶ **Two grids, two representations: the rows are not comparable. A ladder of ideas is not a ranking of results — Deck 3.6 runs one that is.**

Source: Recorded by the book's script for ch. 12; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2; visit-weighted; network seeds 1, 2 from the chapter. Continuous model: Session 3 lab, block B.

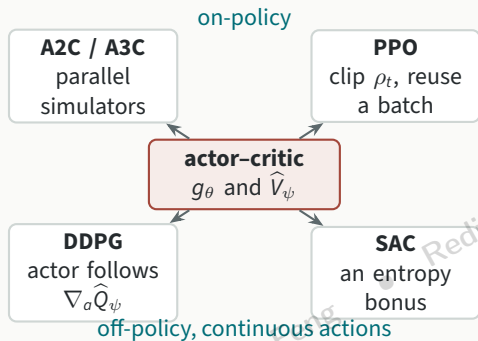
Why Economists Should Care: Actor–Critic as Two-Step Estimation

	Two-step estimation	Actor–critic
nuisance parameter	first stage	the critic — five numbers in Deck 3.2: $E[\delta_t \nabla_{\psi} \hat{V}_{\psi}(s_t)] = 0$
target parameter	second stage	the actor — three numbers: zero update at the optimum
timing	stage one, then stage two	interleaved on every sample

- ▶ Deck 3.2 ran a two-step estimator before it had the name: **eight numbers, one moment condition each**. The stages interleave, so the critic must move faster — in the worked period it moved four hundred times faster.
- ▶ **The saving rule is the estimand; the value function is the nuisance that makes it estimable.**

Source: W. Newey, D. McFadden, *Handbook of Econometrics* 4, 1994, ch. 36; Miao & Feng, ch. 12.

Four Modern Variants, Four Standing Difficulties



- ▶ All four keep an actor and a critic; only the objective or the reuse of data changes. PPO's ratio is Deck 3.3's and its advantage is Deck 3.4's λ -return; PPO also aligns language models (Session 5).
- ▶ None of them removes this deck's four bills: 10^5 steps for 40 states, seed reversals, early lock-in, a last iterate that was not the best.
- ▶ **Grade against an exact solution and report seeds as a range. Next, Deck 3.6: every rung, one model, one protocol.**

Source: Mnih et al., *ICML*, 2016; Schulman et al., 2017; Lillicrap et al., *ICLR*, 2016; Haarnoja et al., *ICML*, 2018; P. Henderson et al., *AAAI*, 2018.

Session 3.6

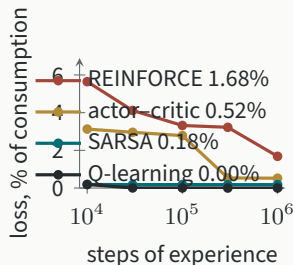
The Growth Model Across Algorithms

The full table Deck 3.2 opened; then actor-critic on the continuous model, and the bridge to Session 4



Zhigang Feng

Every Method of the Session, on One Model and One Grade

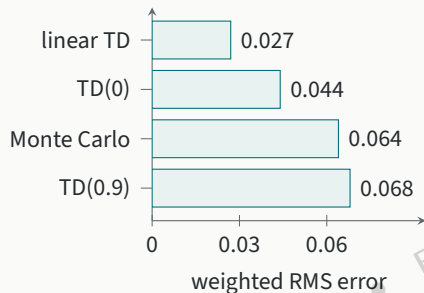


Method	Estimates	Loss: weighted / all
MC, exploring starts	Q^g	- / 0.20%
SARSA	best ϵ -soft rule's value	0.18% / 0.38%
Q-learning	Q^*	0.00% / 0.01%
REINFORCE	the parameters θ	1.68% / 2.10%
actor-critic	θ and V^{g_θ}	0.52% / 1.24%

- **One instance, every row and every curve:** the recorded **tabular** run on the 7×2 grid — 14 states, 20 seeds, 10^6 steps. Four of the five are on the figure.
- **Replaced:** the expectation by samples, the table by parameters, the max by a gradient step.

Source: Recorded by the book's script; tabular, 7×2 grid, 10^6 steps (MC: 1.8×10^6). Miao & Feng, ch. 12.

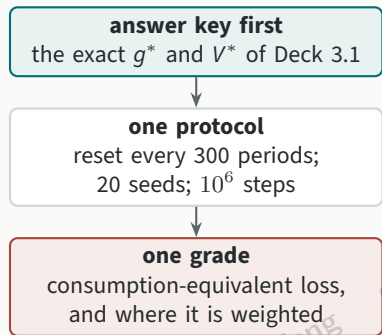
Five Readings You Could Not Make in Deck 3.2



- ▶ **Sampling leads early, bootstrapping late:** at 10^3 steps Monte Carlo is at 0.596, TD(0) at 1.015.
- ▶ **Monte Carlo control needs exploring starts** — greedy covers 3 cells of 68 and loses 2.73%. **SARSA's 0.18% is not an error:** it is the best ϵ -soft rule's own value.
- ▶ **The critic is the baseline:** actor-critic 0.52% against REINFORCE's 1.68%.
- ▶ **Deck 3.2's demo is on none of these rows** — continuous model, 0.014%.

Source: Recorded by the book's script for ch. 12, tabular, 7×2 grid, $\beta = 0.95$; evaluation errors weighted by $\bar{d}$, medians of 20 paths. Bars at 10^5 steps, except the linear fixed point. Last bullet: the continuous demo of Deck 3.2.

The Protocol Is the Part That Travels



- ▶ Every number is a distance from a benchmark built before any learner ran.
- ▶ **Say where the grade is taken, and on what.** Tabular, 7×2 : 0.52% where the economy lives, 1.24% over all 14 states. The continuous demo: 0.014% and 0.015%.
- ▶ **Report medians and seed counts.** That run's best seed is 0.35%, worst 2.67%; the demo's, 0.002% and 0.106%.
- ▶ **Session 4 has no closed form: its answer key must be built classically first.**

Source: Tabular run recorded by the book's script, 7×2 grid, 20 seeds; the continuous demo from Session 3 lab, block A2.

Session 3.6

Deep Value Iteration

Actor-critic with a known model, on Session 2's continuous growth model



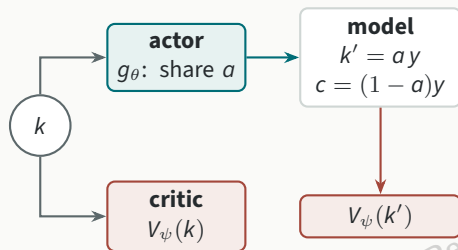
From the Grid to a Network

	Value function iteration	Deep value iteration
states	a fixed grid	a fresh sample every step
value	a table, interpolated	a network $V_{\psi}(k)$
improvement	exact $\max_{k'}$ at each point	a policy network, by gradient ascent
cost in dimension d	N^d	grows with the sample
convergence	a contraction	no general guarantee

- ▶ The capital stock is now continuous: no table of states, no list of actions to compare.
- ▶ **The trade: give up the exact $\max$ and the contraction; gain reach in dimension.**

Source: Maliar, Maliar & Winant, *JME* 122, 2021; Miao & Feng, *AI for Economists*, ch. 12, “When DP, when RL”.

Two Networks: an Actor and a Critic, with the Model Known



the actor's gradient passes
through the model and the critic

- ▶ **Actor**: Session 2's saving-share network.
Critic: a value network.
- ▶ The model is known and differentiable: the actor takes a **pathwise gradient** of $u(c) + \beta V_\psi(k')$ through the budget constraint.
- ▶ Deck 3.5's score-function gradient was for unknown models. This one has far less variance.
- ▶ **Same two roles as Deck 3.5. Only the source of the actor's gradient changed.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "Actor-critic for the household problem"; Session 3 lab, block F.

The Value Step, and Why the Target Is Detached

- ▶ Hold the actor fixed. One Bellman backup under it is the label:

$$y(k) = u(c_{\theta}(k)) + \beta V_{\psi}(k'_{\theta}(k)).$$

- ▶ Fit the critic by least squares on sampled states: $\min_{\psi} \frac{1}{n} \sum_i (V_{\psi}(k_i) - y(k_i))^2$. Session 2's supervised learning, with a Bellman target as the label.
- ▶ **Detach** the target: no gradient flows through y . Otherwise the network lowers the loss by moving the label towards the prediction.
- ▶ An exact backup shrinks the value error by β . A fitted backup adds a regression error, every round.
- ▶ **The network being trained produces its own label: bootstrapping, as in TD(0), without sampling noise.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “Neural networks and DQN”; V. Mnih et al., *Nature* 518, 2015.

The Policy Step: Climb the Right-Hand Side of the Bellman Equation

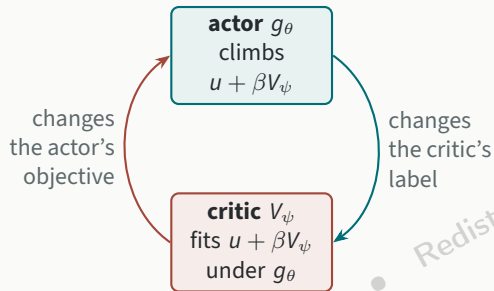
- ▶ Hold the critic fixed. Improve the actor by gradient *ascent* on

$$\max_{\theta} \frac{1}{n} \sum_i \left[u(y_i - k'_{\theta}(k_i)) + \beta V_{\psi}(k'_{\theta}(k_i)) \right], \quad y_i = Ak_i^{\alpha} + (1 - \delta)k_i.$$

- ▶ A grid solver compares every k' . Here a gradient step replaces the max; no inner optimisation is solved.
- ▶ Structure is built into the actor, not penalised: the sigmoid share keeps $c > 0$ and $k' > 0$ at every θ .
- ▶ The actor needs the critic's *slope*, $V'_{\psi}(k')$, not its level. A critic can fit levels well and slopes badly.
- ▶ **Two networks, because the max over a network's output is itself a hard problem. The actor amortises it.**

Source: Session 3 lab, block F; Session 2.5, "Feasibility by Construction".

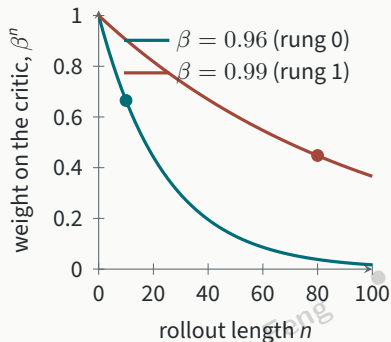
Two Moving Targets



- ▶ Each network's target depends on the other. Neither solves a fixed problem.
- ▶ The pair can agree and still be wrong: a critic that correctly values a bad actor satisfies its own Bellman equation.
- ▶ Practice: let the critic move faster than the actor; start both where the answer is known.
- ▶ **Here that is the steady state:**
 $V(k^*) = u(c^*) / (1 - \beta)$ **fixes the critic's level.**

Source: V. Konda, J. Tsitsiklis, "Actor-critic algorithms", *NeurIPS* 12, 2000; Session 3 lab, block F.

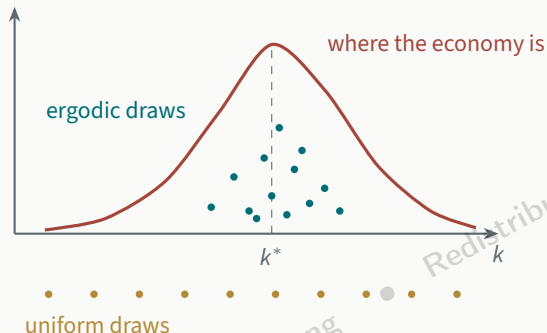
One Step or Many



- ▶ Roll the actor n periods through the known model, then ask the critic:
 $J_n(k) = \sum_{t < n} \beta^t u(c_t) + \beta^n V_\psi(k_n)$.
- ▶ $n = 1$: one-step fitted value iteration.
Larger n leans less on the critic: its weight is β^n .
- ▶ With shocks, long rollouts add variance. This model is deterministic, so they add none.
- ▶ **The lab:** $n = 10$ on rung 0, $n = 80$ on rung 1. Critic weights 0.66 and 0.45.

Source: Deck 3.4, n -step returns; Session 3 lab, block F (Session 2's rungs 0 and 1).

Where to Sample



- ▶ The sampling distribution weights the critic's regression: accuracy is bought where states are drawn.
- ▶ **Ergodic sampling**: simulate under the current actor, train where the economy goes. Efficient, and blind to rare states.
- ▶ The lab keeps Session 2's band, uniform in $\log k$ on $[0.2, 2] \times k^*$; off-domain errors are reported apart.
- ▶ **A solution is claimed only where it was trained and tested.**

Source: Schematic. Judd, Maliar & Maliar, *Quantitative Economics* 2(2), 2011; Session 2.5, "Where to Sample".

The Code: One Training Step

```
def rollout_objective(actor, critic, k, n_step):          # J_n(k)
    crra, beta = rung.gamma, rung.beta                  # curvature, discount
    total, disc, z = 0.0, 1.0, torch.ones_like(k)
    for _ in range(n_step):
        k_next = actor(k, z)                            # feasible by construction
        total = total + disc * utility(actor.resources(k, z) - k_next, crra)
        disc, k = disc * beta, k_next
    return total + disc * critic(k)
loss_actor = -rollout_objective(actor, critic, k, n).mean() # climb J
loss_actor.backward(); opt_actor.step()
with torch.no_grad():                                   # detach the label
    target = rollout_objective(actor, critic, k, n)
loss_critic = (critic(k) - target).square().mean()
loss_critic.backward(); opt_critic.step()
```

- ▶ The listing keeps Session 2's beta for the discount, crra for the curvature.
- ▶ **No benchmark enters either loss; the closed form is kept for grading.**

Source: Session 3 lab, block F; the training step of Miao & Feng, *AI for Economists*, ch. 12.

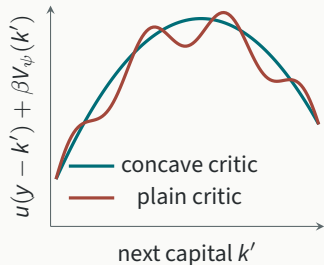
Session 3.6

Structure, and the Choice of Road

What a concave critic buys, and how to tell a right answer from a quiet residual



Value Functions Have a Shape, and a Critic Can Be Built to Keep It



- ▶ Concave utility and a convex constraint set make V increasing and **concave**; the Bellman operator preserves both.
- ▶ A plain network knows none of it, and the actor climbing $u + \beta V_\psi$ follows every bump.
- ▶ An **input-concave** critic (Session 2.6's input-convex network, negated) holds the shape at every ψ .
- ▶ **Then the actor's objective has one peak, found by gradient ascent from any start.**

Source: Schematic. Stokey, Lucas & Prescott, *Recursive Methods*, 1989, §4.2; Amos, Xu & Kolter, *ICML*, 2017; Session 2.6.

What That Buys, What It Does Not, and What It Costs

- ▶ Fitted value iteration is a backup, then a regression: $V_{n+1} = \Pi(TV_n)$, with Π the fit onto the network class.
- ▶ T is a contraction. ΠT need not be, and concavity of the class does not make it one: a projection after a contraction is still a projection. That is the “no general guarantee” row.
- ▶ What the class does buy: the label TV_n stays inside it, so the fit never bends a target into a shape it cannot take, and the actor never chases a bump the critic invented.
- ▶ The cost is measured: 7.8 times the RMSE on Session 2.6’s test. It must come from theory: if a discrete choice kinks the true value, concavity is bias.
- ▶ **This lab’s critic is unconstrained; the gates protect it. Buy structure when theory supplies it, and test as if you had not.**

Source: G. Gordon, “Stable function approximation in dynamic programming”, *ICML*, 1995; Session 2.6.

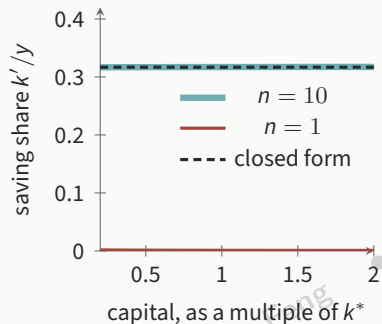
Euler or Bellman: Measured on Session 2's Rungs

held-out states	rung 0		rung 1	
	Euler	Bellman	Euler	Bellman
max policy error (gate: 1%)	0.64%	0.14%	0.04%	0.25%
max value error, % of consumption (gate: 0.1%)	–	0.07%	–	0.08%
simulated against analytical k^* (gate: 1%)	0.47%	0.12%	0.18%	0.32%
$\log_{10}$ max Euler error	–1.95	–2.78	–3.34	–2.20
seconds, one CPU core	1.4	19	4.6	165

- ▶ Euler: one network, a sharper condition, no value. Bellman: two networks, no first-order conditions needed (kinks, discrete choices), and a value for welfare comparisons.
- ▶ **A decision, not a doctrine: take the Euler road when the first-order conditions are clean.**

Source: Session 2 lab, block D; Session 3 lab, block F. Session 2 calibration: $\alpha = 0.33, A = 1$; rung 0: $\beta = 0.96, \delta = 1$, log utility; rung 1: $\beta = 0.99, \delta = 0.025$, CRRA 2.

A Small Residual Is Not a Right Answer



rung 0, same seed, same steps	$n = 10$	$n = 1$
$\log_{10}$ max Bellman residual	-1.77	-1.78
max policy error	0.14%	99.7%
steady-state gap	0.12%	91%

- ▶ With $n = 1$ the critic values a bad actor *correctly*: that pair satisfies its own Bellman equation too.
- ▶ **The residual cannot tell them apart. The benchmark and the steady state can.**

Source: Session 3 lab, block F (Session 2 rung 0); Miao & Feng, ch. 12, “Knowing whether the answer is right”.

Session 3.6

From One Household to an Economy

When to use what, and what Session 4 inherits



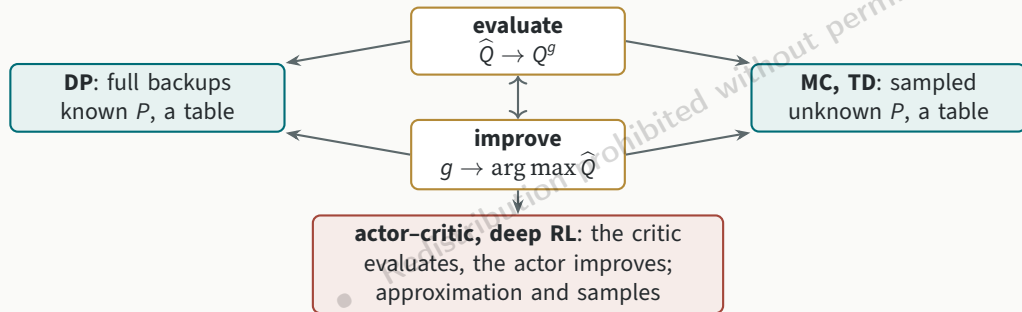
When Dynamic Programming, When Reinforcement Learning

	model known	model unknown
low dimension	DP: nothing beats it	tabular RL; coverage is the issue
high dimension	approximate DP: deep value iteration	deep RL's home ground

- ▶ Bottom left is where most structural economics lives: the model can be simulated, the grid cannot be built.
- ▶ Already in use: dynamic discrete choice, where Rust's estimation re-solves the Bellman equation inside the parameter search; high-dimensional macro; interest-rate and tax rules; and platform pricing, where the environment holds other agents' responses.
- ▶ **The caveat: reinforcement learning scales the *solution*, not the *identification*. The data discipline lives in the calibration.**

Source: J. Rust, *Econometrica* 55(5), 1987; Hinterlang & Tänzler, Bundesbank DP 51/2021; Miao & Feng, ch. 12, "When DP, when RL".

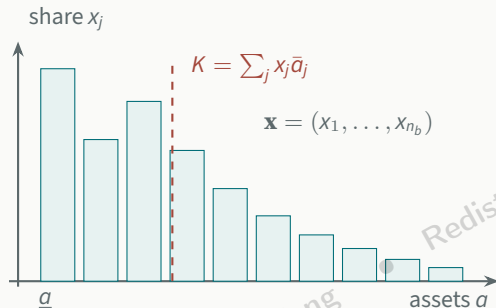
One Backbone: Evaluate, Improve



- ▶ VFI: explicit sums, pure greedy. MC control: sample means, ϵ -greedy. SARSA, Q-learning: bootstrapped samples. Actor-critic: both steps on every transition.
- ▶ **Deck 3.2 opened with this loop and one method on it. Here it is with every method of the session attached.**

Source: After Miao & Feng, *AI for Economists*, ch. 12 (the algorithmic backbone); Sutton & Barto, 2018, §4.6.

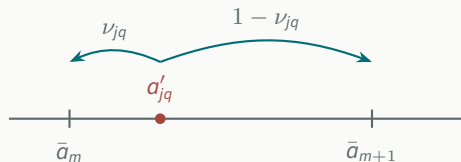
Bridge (1): Put the Distribution in the State, as a Histogram



- ▶ The method starts by choosing what to carry of the wealth distribution. Krusell and Smith carry its mean.
- ▶ Carry a **histogram** instead: shares x_j in n_b asset bins. No shape assumed; the borrowing-limit mass and the tail survive.
- ▶ Prices follow inside the model: K , then r and w from the firm's conditions.
- ▶ **The state is $n_b + 2$ numbers, which a network takes as one input vector.**

Source: Schematic after Miao & Feng, ch. 12; method of Feng, Han, Sargent & Zhu (2026, working paper), research in progress.

Bridge (2): an Analytic Transition Matrix, Not a Simulated Panel



mass is split between neighbouring bins in proportion to distance

$$\mathbf{x}_{t+1} = \mathbf{x}_t \hat{\mathbf{M}}_g(\mathbf{x}_t)$$

- ▶ Households in bin j with shock node z_q choose a'_{jq} ; its mass is split between the two neighbouring bins, and summing over nodes gives a transition matrix $\hat{\mathbf{M}}_g$. Mass is conserved exactly.
- ▶ The split is piecewise linear in a' , so $\mathbf{x}_{t+1}$ is *differentiable* in the policy's parameters. A simulated panel is not.
- ▶ **This is the discrete-time answer to “ Γ_g has no closed form” that Deck 3.1 promised.**

Source: The two-point split is E. Young, *JEDC* 34(1), 2010. Using it as an analytic, differentiable law of motion a learner can train against is Z. Feng, J. Han, T. Sargent & S. Zhu (2026, working paper). Miao & Feng, ch. 12, “An analytic transition matrix”.

Bridge (3): the Household's Actor-Critic

Household problem, no aggregate shocks

- 1 initialise critic $V_\psi(a, z, \mathbf{x})$, actor $a' = g_\theta(a, z, \mathbf{x})$, target copy $\psi^- \leftarrow \psi$
- 2 **repeat:** draw a batch of distributions $\mathbf{x}$ (perturbed, and pushed forward under the actor)
- 3 prices: $K = \sum_j x_j \bar{a}_j$, then r, w ; choices a' and c at every $(\bar{a}_j, z_q)$
- 4 $\mathbf{x}' = \mathbf{x} \hat{\mathbf{M}}_g$, **held constant when differentiating**
- 5 critic step: regress on $y = u(c) + \gamma \sum_q p_q V_{\psi^-}(a', z_q, \mathbf{x}')$
- 6 actor step: pathwise gradient of $u(c) + \gamma \sum_q p_q V_\psi(a', z_q, \mathbf{x}')$; sync ψ^- at intervals
- 7 **until** Euler residuals are small *and* forward-simulated aggregates stop moving
 - Block F with a longer input. Line 4 is the economics: an atomistic household does not count its own effect on $\mathbf{x}'$.
 - **A method, shown without results: the research is in progress. Session 4 owns equilibrium, the government, and validation.**

Source: Miao & Feng, *AI for Economists*, ch. 12; Feng, Han, Sargent & Zhu (2026, working paper), research in progress.

The Six Things Deck 3.2 Used on Credit, and What Each One Cost

The idea, used on credit	Paid	What the answer cost on this model
sample the expectation	3.3	greedy covers 3 cells of 68 and loses 2.73%; exploring starts, 0.20%
bootstrap one step	3.4	the step size alone moves TD(0) from 1.270 to 0.044
approximate the value	3.5	three bases: same weighted error, maximum error 14 times apart
improve by a gradient	3.5	tabular, 7×2 : REINFORCE 1.68%, actor-critic 0.52%
evaluation inside improvement	3.5	with a network, no guarantee; the critic must run 400 times faster
an answer key, first	3.1, 3.6	the protocol: where the grade is taken, and medians over seeds

- ▶ Deck 3.2 also found one thing the plan did not expect: **rolling the demo out further made it worse**, at every step size tried.
- ▶ **Six debts, six prices, one unit. What is still owed is an equilibrium, a government, and a test that is not a residual.**

Source: Decks 3.3–3.5, 7×2 grid unless stated; the continuous demo from Session 3 lab, block A2.

This Was an Illustration

Built in Session 3

an answer key before any learner
actor and critic, model known
pathwise gradients
a grade in units of consumption
small residual, not a right answer

Used in Session 4

the stationary equilibrium by classical methods first
household policy and value, distribution as an input
only the government differentiates through $\mathbf{x}'$
consumption-equivalent welfare across tax regimes
judge a solution by its simulated aggregates

- ▶ **Every number this session was on a growth model chosen so that an answer key exists** — a printable grid, a 400-point dynamic programme, a closed form. The point was to watch each idea get *graded*; none of it is a finding about an economy.
- ▶ **Session 4 starts on the problems these tools are for: Aiyagari, Krusell–Smith, transitions, taxation — where no answer key exists and a solution is judged by its simulated aggregates.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “Knowing whether the answer is right”.